

Cyber Range Capstone — Honeypot Lab Checklist

MySQL Server Security Monitoring & Threat Detection Lab

Defensive Lifecycle: Build hard → Instrument → Baseline → Detect →  Weaken & Expose

Architecture: Windows VM + MySQL →  MDE + Sentinel + MySQL Audit Logs

Complete Lifecycle: Build • Harden • Monitor • Baseline • Detect • Expose • Investigate • Respond

Overview

This checklist follows the complete defensive lifecycle of an internet-exposed asset: **build, harden, monitor, baseline, detect, expose, investigate, and respond**.

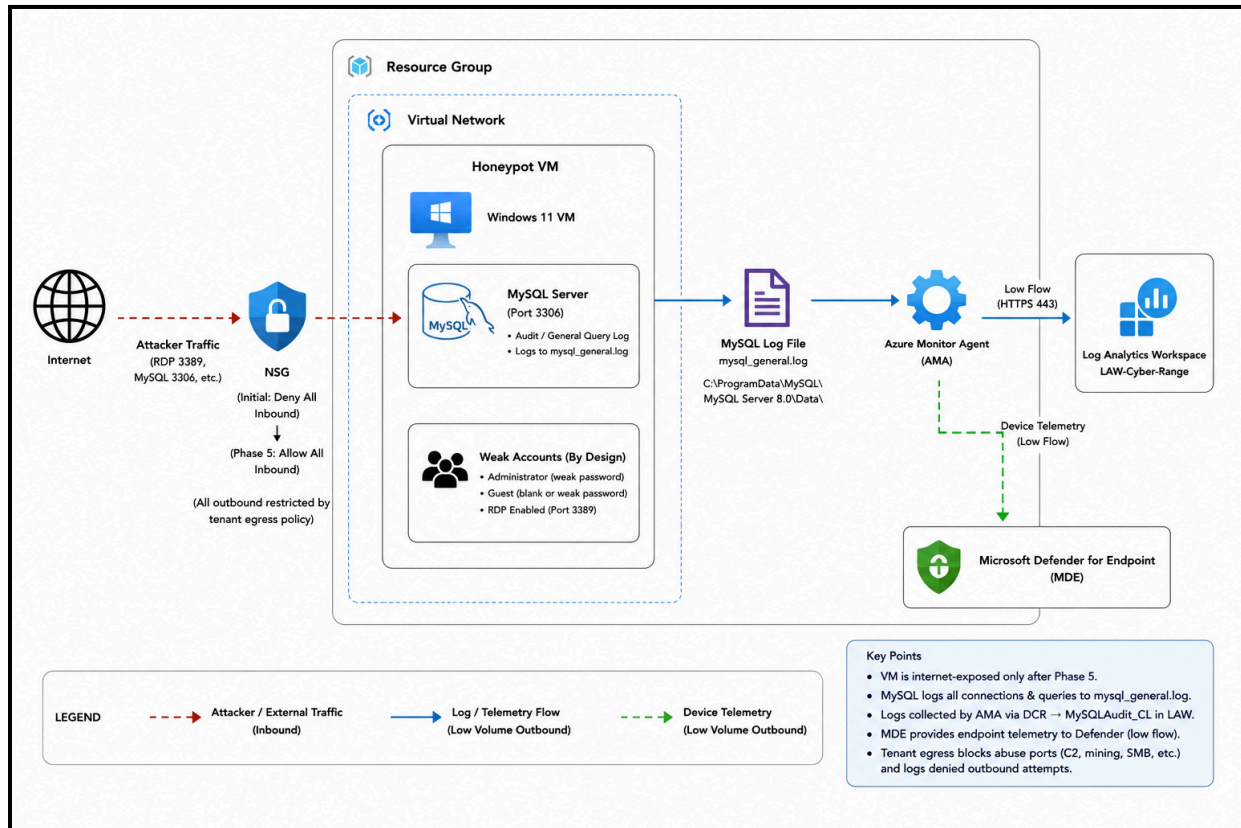
Start by securing the system, enabling comprehensive logging, and establishing a clean baseline. Develop and validate your detection rules **before** exposing the asset to the internet. Once monitoring and detections are in place, deliberately weaken and expose the environment to generate realistic attacker activity.

Follow each phase in order. **The goal is to have visibility and detections ready before the attack begins, allowing you to detect, investigate, and document the resulting incident from start to finish.**

Phase 0 — Honeypot Architecture

The honeypot architecture consists of:

- **Resource Group** containing a Virtual Network with the Honeypot VM
- **Windows 11 VM** running MySQL Server (Port 3306)
- **Network Security Group (NSG)** initially denying all inbound traffic
- **Microsoft Defender for Endpoint (MDE)** for endpoint telemetry
- **Azure Monitor Agent** for log collection
- **Log Analytics Workspace** to centralize MySQL audit logs
- **Microsoft Sentinel** for detection rules and alerting
- **Weak Accounts** intentionally created during Phase 5 (Administrator, Guest)



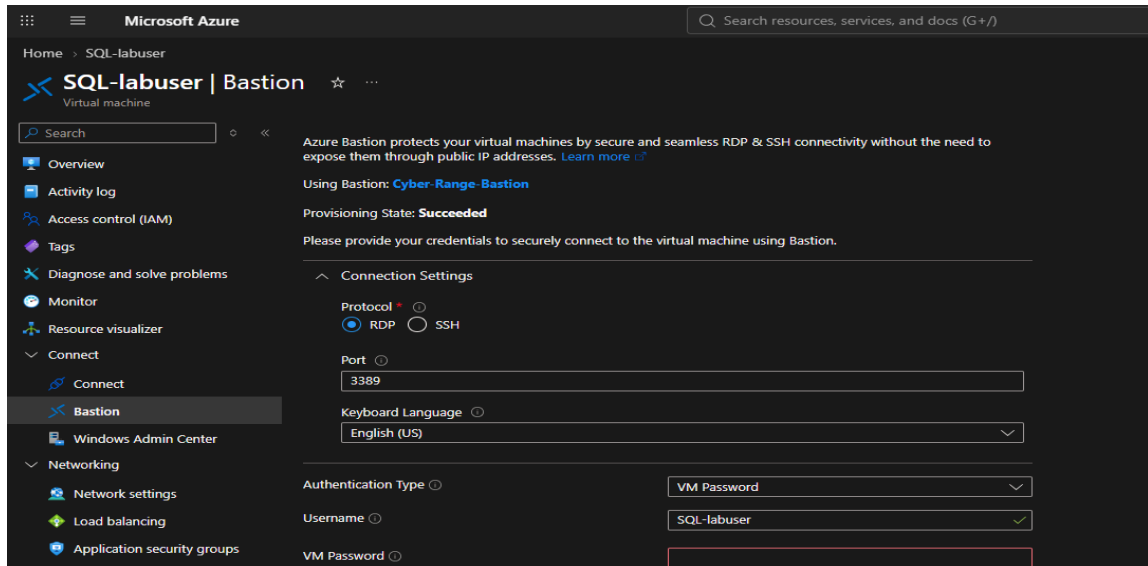
ENVIRONMENT NOTE

Workspace target is LAW-Cyber-Range. Tenant egress is heavily restricted centrally (allow-listed ports, known abuse ports explicitly denied), so a breached box is contained by design — attacker C2 / mining / pivoting attempts are blocked and logged rather than succeeding. Plan detections around denied / attempted outbound, not successful exfil.

Phase 1 — Build the VM Honeypot (locked down while we build it)

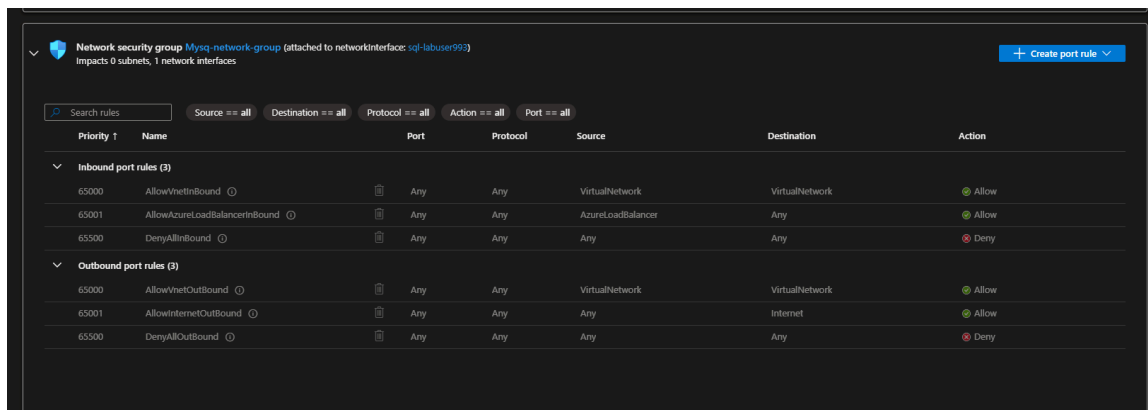
□ Deploy a Windows 11 VM in your own resource group

- Name the VM something that looks legitimate, like **“SQL-labuser”** (not “lab_test_1”)



☐ Deny all inbound traffic from the internet

- Configure the Network Security Group to block all inbound traffic initially
- We'll selectively open ports later during Phase 5



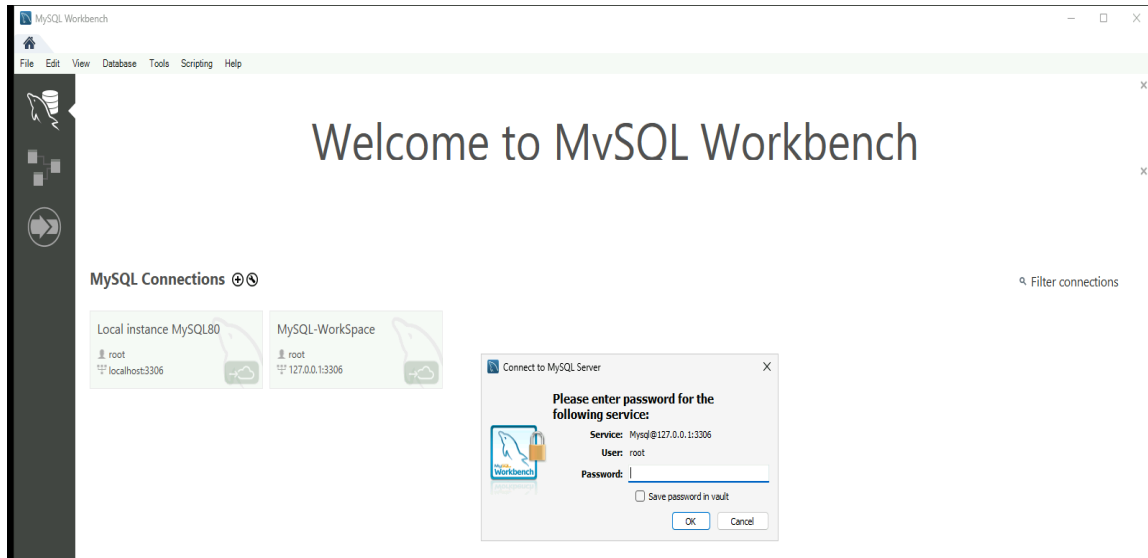
☐ [Onboard the VM to Microsoft Defender for Endpoint \(MDE\)](#), ensure it is showing up in the [DeviceInfo](#) table

	TimeGenerated	Timestamp	DeviceId	DeviceName	ClientVersion	PublicIP	OSArchitecture	OSPlatform	OSBuild	
☐	>	Sep 1, 2026 9:29:33	Sep 1, 2026 9:29:35 PM	07cecec58d3836d6b...	sql-labuser	10.8830.26200.9168	(=) 20.114.175.165	64-bit	Windows11	26200
☐	>	Sep 1, 2026 8:29:33	Sep 1, 2026 8:29:35 PM	07cecec58d3836d6b...	sql-labuser	10.8830.26200.9168	(=) 20.114.175.165	64-bit	Windows11	26200
☐	>	Sep 1, 2026 10:30:...	Sep 1, 2026 10:30:11 PM	07cecec58d3836d6b...	sql-labuser	10.8830.26200.9168	(=) 20.114.175.165	64-bit	Windows11	26200
☐	>	Sep 2, 2026 8:35:1...	Sep 2, 2026 8:35:18 AM	07cecec58d3836d6b...	sql-labuser	10.8830.26200.9168	(=) 20.114.175.165	64-bit	Windows11	26200
☐	>	Sep 2, 2026 8:35:2...	Sep 2, 2026 8:35:27 AM	07cecec58d3836d6b...	sql-labuser	10.8830.26200.9168	(=) 20.114.175.165	64-bit	Windows11	26200

Phase 2 — Install & populate MySQL

- ☐ On your VM, install [Microsoft Visual C++ 2019 Redistributable Package \(x64\)](#) (My SQL Requirement) ([ref](#))
 - On your VM Install [MySQL](#) ([ref](#)), Install with “Developer Default” or “Full” so it installs Workbench/the GUI
 - Install with all defaults

- For the MySQL **root password**, on this phase, make it something strong, store it in a password manager for now (or somewhere you feel comfortable)
 - Everything else Defaults
- ☐ Start MySQL Workbench after the installation, create a new connection, and connect to MySQL
- ☐ Start MySQL Workbench and create a connection

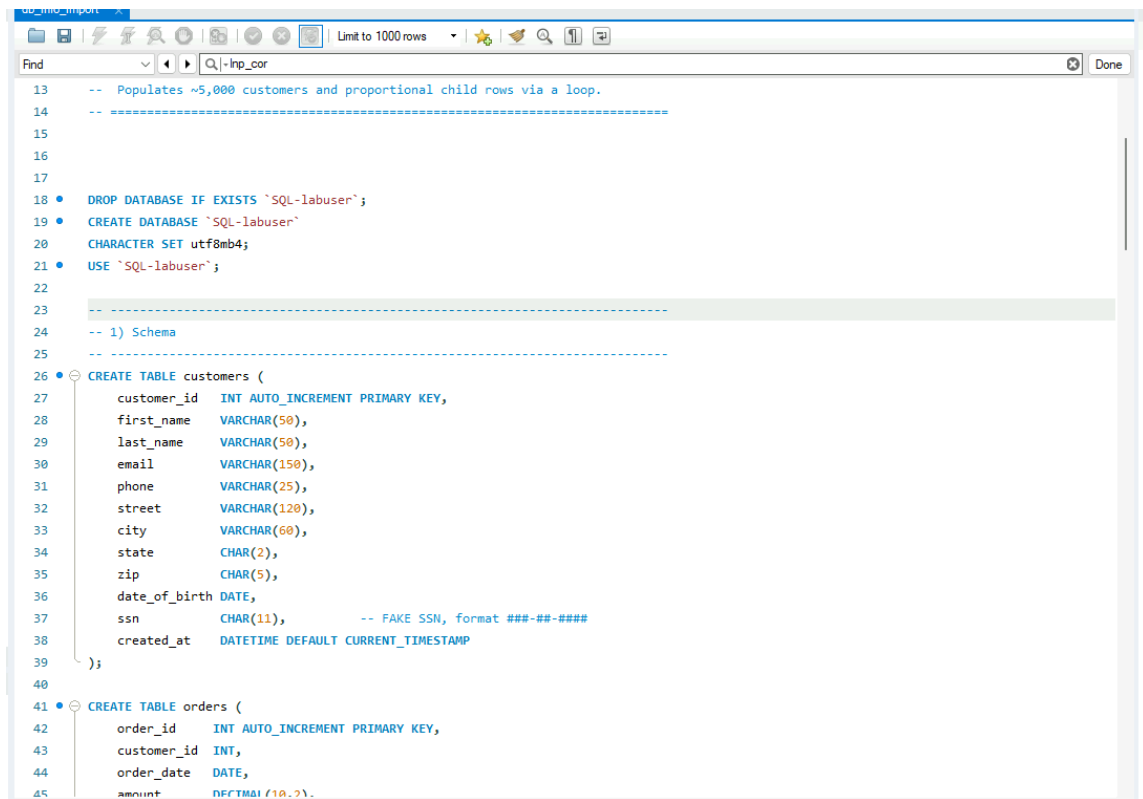


Create a MySQL Database and Import Dummy Data

Create a dedicated MySQL database and populate it with **dummy, non-production data** for the security monitoring and threat-hunting lab.

1. Download the [db_info_import.sql](#) file to the VM.
2. Open the SQL script using MySQL Workbench or another MySQL client.
3. Review the script to confirm that it contains only authorized lab data.
4. Execute the SQL script against your MySQL instance.
5. Verify that the database, tables, and sample records were successfully created and imported.

Lab Note: Use only synthetic or dummy data in this environment. Do not import real credentials, payment-card information, personal data, or other sensitive production data.



```

13 -- Populates ~5,000 customers and proportional child rows via a loop.
14 -- =====
15
16
17
18 • DROP DATABASE IF EXISTS `SQL-labuser`;
19 • CREATE DATABASE `SQL-labuser`
20   CHARACTER SET utf8mb4;
21 • USE `SQL-labuser`;
22
23 -----
24 -- 1) Schema
25 -----
26 • CREATE TABLE customers (
27   customer_id INT AUTO_INCREMENT PRIMARY KEY,
28   first_name VARCHAR(50),
29   last_name VARCHAR(50),
30   email VARCHAR(150),
31   phone VARCHAR(25),
32   street VARCHAR(120),
33   city VARCHAR(60),
34   state CHAR(2),
35   zip CHAR(5),
36   date_of_birth DATE,
37   ssn CHAR(11), -- FAKE SSN, format ###-##-####
38   created_at DATETIME DEFAULT CURRENT_TIMESTAMP
39 );
40
41 • CREATE TABLE orders (
42   order_id INT AUTO_INCREMENT PRIMARY KEY,
43   customer_id INT,
44   order_date DATE,
45   amount DECIMAL(10,2);

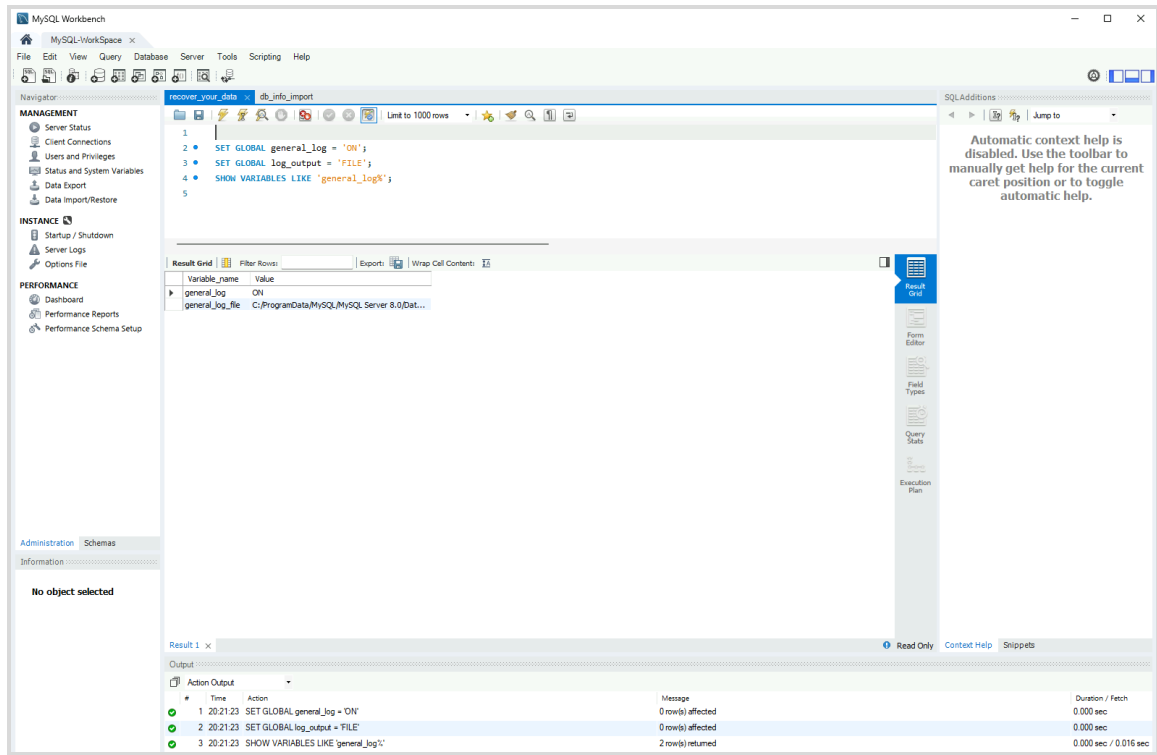
```

- ☐ ⚠ this will seem like MySQL Workbench has frozen, it will just take some time. If the connection drops or it fails, just enter the MySQL password if needed and try again
→ change users from 5000 to 1000 or less if it fails too many times
- ☐ On the Schemas tab, refresh it to observe the new schema (database) `SQL-labuser`
- ☐ **Enable MySQL audit / general query logging** so every connection (success and failure) and query is written to a log file. Do that by running this query:

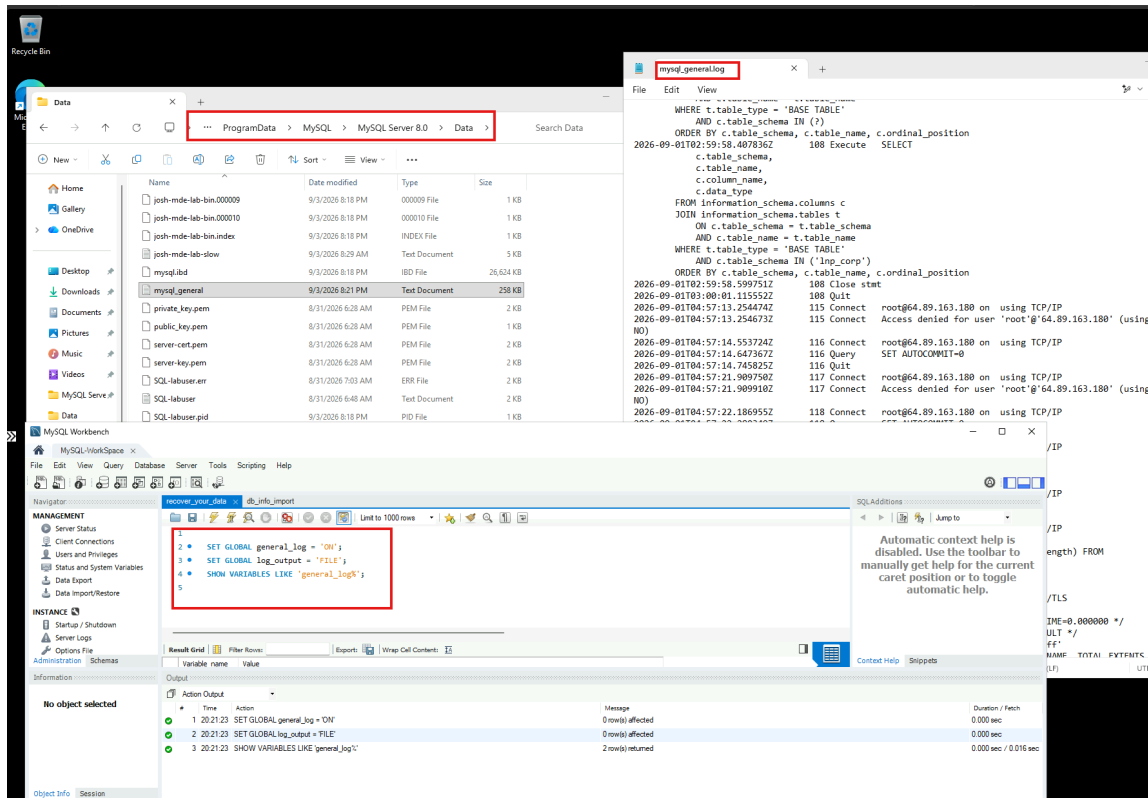
```

SET GLOBAL general_log = 'ON';
SET GLOBAL log_output = 'FILE';
SHOW VARIABLES LIKE 'general_log%';

```



- ☐ Download [my.ini](#) and replace this file on your VM with it: **C:\ProgramData\MySQL\MySQL Server 8.0\my.ini**
- ☐ This file tells MySQL to log everything to, it also allows login over the network:
"C:\ProgramData\MySQL\MySQL Server 8.0\Data\mysql_general.log"



- ☐ Restart the MySQL80 service (services.msc)
- ☐ Run a few “SELECT” queries and then check the mysql_general.log to ensure the logs are coming through.
- ☐ Note the MySQL log file path (needed for the DCR in Phase 3)

Phase 3 — Wire logging to Log Analytics

- ☐ Ensure your VM is ON and RUNNING, this is important
- ☐ Confirm device telemetry is flowing to LAW-Cyber-Range (check the DeviceInfo Table)
- ☐ Create a custom-text-log DCR pointing AMA at the MySQL log file → lands in a custom table: MySQLAudit_CL
- ☐ It's VERY important this is correct (Logs may take up to 30 minutes to appear)

File pattern: **C:\ProgramData\MySQL\MySQL Server 8.0\Data\mysql_general.log**

Table name: **MySQLAudit_CL**

Record delimiter: **TimeStamp**

Timestamps format: **ISO 8601**

Home > Monitor | Data Collection Rules > Create Data Collection Rule

Add new data source

Data source Destination Transform (optional)

Select which data source type and the data to collect for your resource(s).

Data source type * Custom Text Logs

File pattern * C:\ProgramData\MySQL\MySQL Server 8.0\Data\mysql_general.log

Table name * MySQLAudit_CL

Record delimiter TimeStamp

Timestamp format ISO 8601 (Example: 2024-10-29T18:28:34)

Save Next: Destinations > Cancel

☐ For the Destination, choose LAW_Cyber_Range:

Home > Monitor | Data Collection Rules > Create Data Collection Rule

Add new data source

Data source **Destination** Transform (optional)

Select the destination(s) where the data will be delivered. Note that you can only select destinations that are in the same region as the Data Collection Rule. Normal usage charges for the destination will occur. [Learn more about pricing.](#)

Data collection rule region: **East US 2**

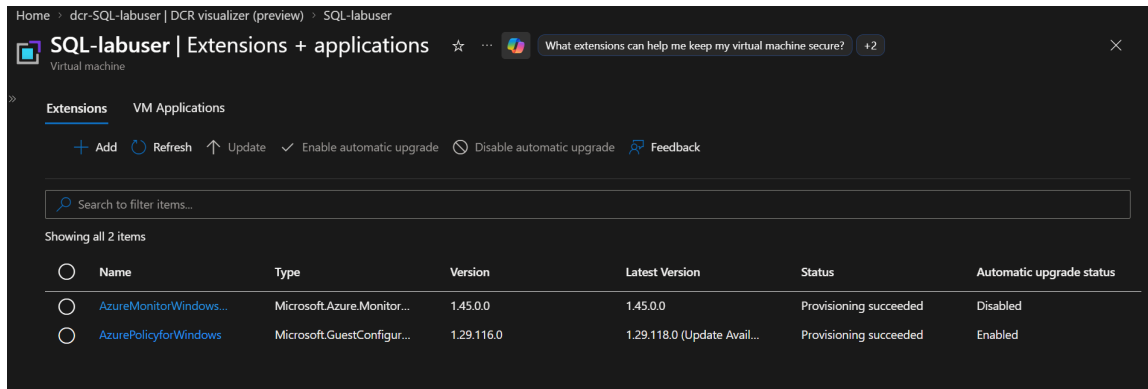
This data source supports only one destination.

+ Add destination Edit destination Delete destination

Destination type	Destination details
☐ Log Analytics Workspaces	LAW-Cyber-Range (Cyber-Range-Admin-SOC)

Save < Previous Cancel

☐ Immediately after successfully creating the DCR, The Azure Monitor Agent will start to be installed on your VM. Browse to your VM → Settings → Extensions + applications, and look for “AzureMonitorWindowsAgent”:



- ☐ Verify ingestion: query MySQLAudit_CL and confirm test connections / queries appear

MySQLAudit_CL

| project TimeGenerated, RawData, _ResourceId

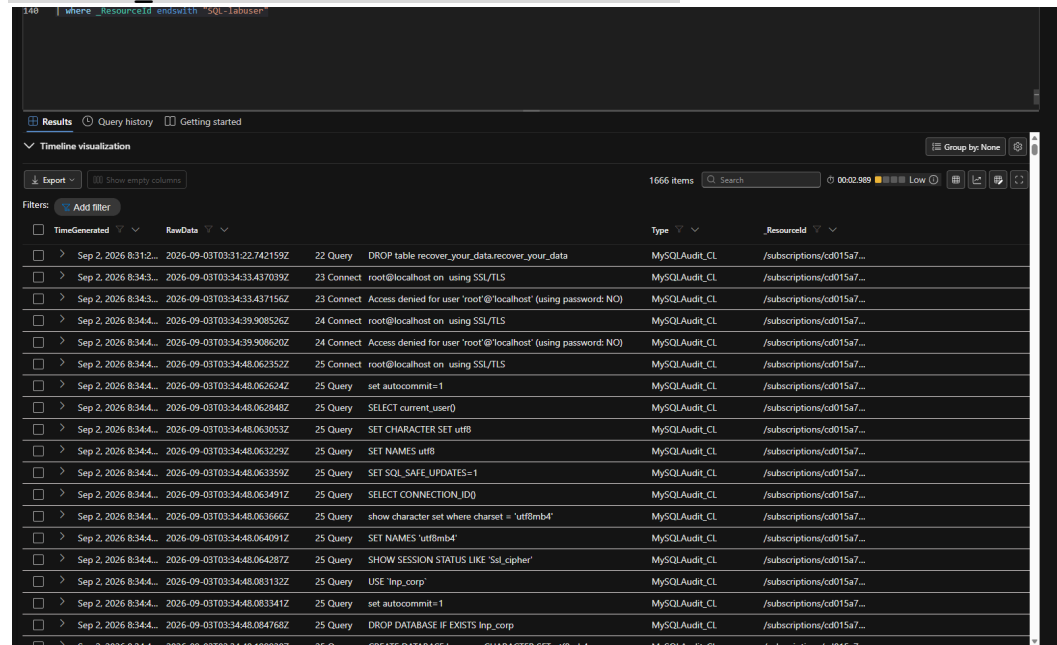
| where _ResourceId endswith "your-VM-Name"

- ☐ Verify core device tables are populating. Once your logs are coming through, you can move on to Phase 4. ⚠ This table contains EVERYONE'S MySQL logs, you have to filter yours to match your Resource

- ☐ For example:

MySQLAudit_CL

| where _ResourceId endswith "SQL-labusers"



Phase 4 — Write detections (while the box is still clean)

Author these as Sentinel analytics rules, before exposure, and confirm they are quiet against the clean baseline (no real successes yet).

Rule — successful logon to the VM: DeviceLogonEvents, ActionType == "LogonSuccess", public RemoteIP

☐ **Create a Sentinel Analytics Rule for successful logons to the VM - Analytics Rule.**
Name: Honeypot-Cyber Defense – Brute Force Detection-Gal

- ☐ ****Description:**** Detects successful and failed logon attempts involving the `administrator` and `guest` accounts on the specified SQL lab virtual machine. The query captures the timestamp, remote IP address, account name, device name, action type, and logon type to help identify suspicious authentication activity such as brute-force attacks, password spraying, or unauthorized access.

```
// Virtual Machine Logons
let MyDevice = "sql-labuser";
DeviceLogonEvents
| where DeviceName == MyDevice
| where AccountName in~ ("administrator", "guest")
| where ActionType in~ ("LogonSuccess", "LogonFailed")
| project Timestamp, RemoteIP, AccountName, DeviceName, ActionType,
LogonType
| order by Timestamp desc
```

The screenshot shows the 'Analytics rule wizard - Edit existing Scheduled rule' interface in Sentinel. The rule name is 'Honeypot-Cyber Defense – Brute Force Detection-Gal'. The description matches the one provided in the checklist. The MITRE ATT&CK categories are 'Credential Access (1)' and 'Impair Process Control (1)'. The severity is set to 'High' and the status is 'Enabled'. The rule query is the same as the one provided. The rule frequency is 'Run query every 10 minutes', the rule period is 'Last 10 minutes data', the rule start time is 'Automatic', and the rule threshold is 'Trigger alert if query returns more than 0 results'. The interface includes a sidebar with steps: General, Set rule logic, Incident settings, Automated response, and Review + create. At the bottom, there are buttons for '< Previous', 'Save', and 'Cancel'.

Fill in Entity Mapping and Analytics Rule settings

Alert enhancement

Entity mapping

Map up to 10 entities recognized by Microsoft Sentinel from the appropriate fields available in your query results. This enables Microsoft Sentinel to recognize and classify the data in these fields for further analysis. For each entity, you can define up to 3 identifiers, which are attributes of the entity that help identify the entity as unique. [Learn more >](#)

Host

HostName

DeviceName

+

Add identifier

IP

Address

RemoteIP

+

Add identifier

+

Add new entity

> Custom details

> Alert details

Query scheduling

Run query every *

10

Minutes

Lookup data from the last *

10

Minutes

Start running ⓘ

☒ Automatically

☐ At specific time (Preview)

6/23/2026

12:00 PM

Rule — successful login to the MySQL database: MySQLAudit_CL, connect / auth-success events

When querying MySQLAudit_CL, you'll notice the logs come in the format of "RawData". These need to be parsed out into columns in order to work with them effectively. You can try to figure it out, or use this query to parse them out:

Create a Microsoft Sentinel Analytics Rule

Create a Microsoft Sentinel Analytics Rule to detect successful logons to the MySQL virtual machine.

Analytics Rule Name:

Cyber-Defense – Successful Login to MySQL-Final-MySQL

Purpose: Detect and alert on successful authentication to the MySQL VM, providing visibility into potentially unauthorized access and supporting subsequent investigation and threat hunting

Rule query:

```
// SQL Server

// SQL Server
let MyDevice = "sql-labuser";
let FailedConnections =
MySQLAudit_CL
| extend RawData = replace_string(RawData, "\t", " ")
| extend DeviceName = tostring(split(_ResourceId, "/")[-1])
| where DeviceName == MyDevice
| where RawData has "Access denied"
| extend ConnectionId = extract(@"^S\s+(\d+)\s+Connect", 1, RawData)
| distinct ConnectionId;
MySQLAudit_CL
//| where TimeGenerated > MyTimeframe
| extend RawData = replace_string(RawData, "\t", " ")
| extend DeviceName = tostring(split(_ResourceId, "/")[-1])
| where DeviceName == MyDevice
| where RawData has "Connect"
| extend ConnectionId = extract(@"^S\s+(\d+)\s+Connect", 1, RawData)
| extend ActionType =
    case(
        RawData has "Access denied", "LogonFailure",
        ConnectionId in (FailedConnections), "Ignore",
        "LogonSuccess"
    )
| where ActionType != "Ignore"
| extend RawData = replace_string(RawData, "\t", " ")
| extend Username =
replace_string(tostring(split(tostring(split(RawData, "@")[0]), " ")[-1]),
"'", "")
```

```
| extend IPAddress = replace_string(tostring(split(split(RawData,"@")[1],  
" ")[0]), "'", "")  
| where ActionType == "LogonSuccess"  
| project TimeGenerated, DeviceName, Username, IPAddress, ActionType,  
RawData  
| order by TimeGenerated desc
```

Fill in Entity Mapping and Analytics Rule settings

Alert enhancement

Entity mapping

Map up to 10 entities recognized by Microsoft Sentinel from the appropriate fields available in your query results. This enables Microsoft Sentinel to recognize and classify the data in these fields for further analysis. For each entity, you can define up to 3 identifiers, which are attributes of the entity that help identify the entity as unique. [Learn more >](#)

IP

Address

RemoteHost

+ Add identifier

Host

HostName

DeviceName

+ Add identifier

+ Add new entity

Custom details

Alert details

Query scheduling

Run query every *

Minutes

Lookup data from the last *

Minutes

Start running ⓘ

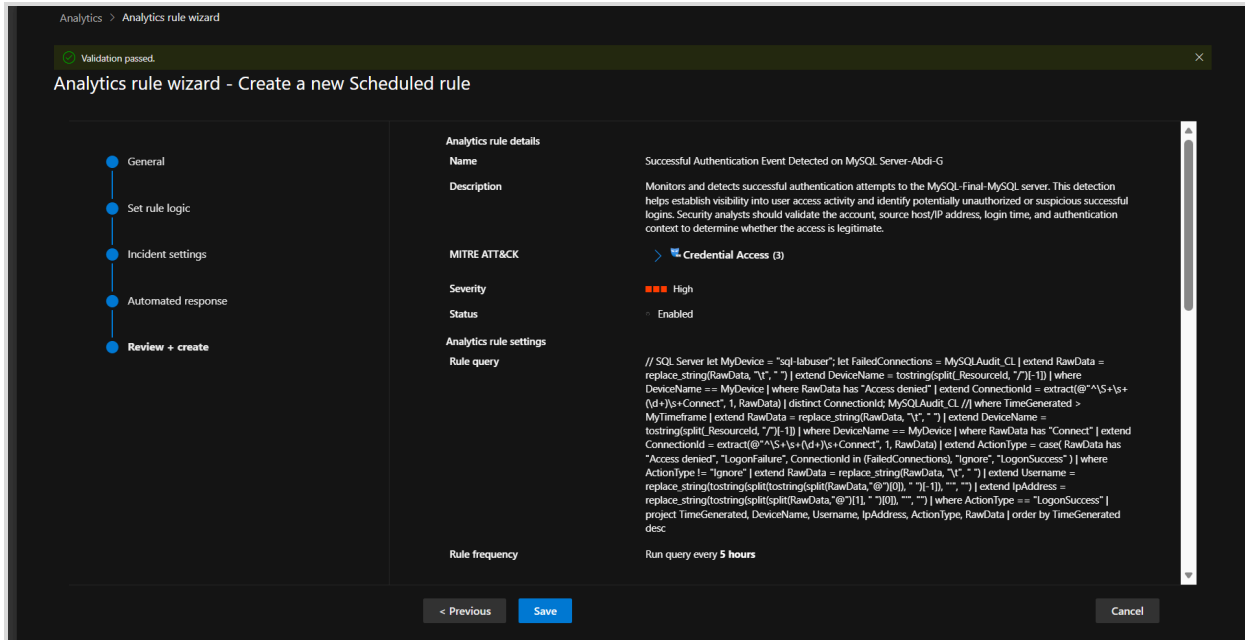
- ☒ Automatically
- ☐ At specific time (Preview)

6/23/2026



12:00 PM

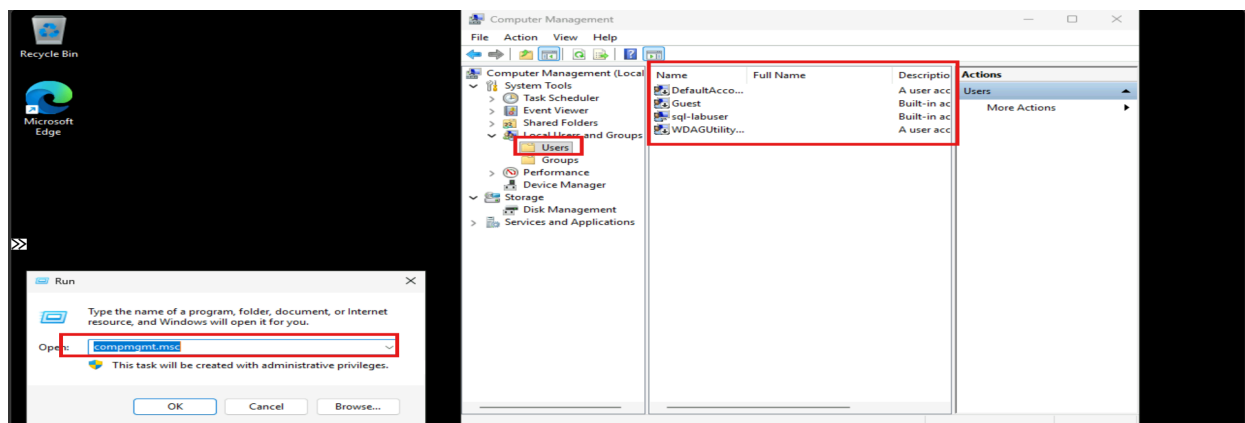
[< Previous](#)[Next : Incident settings >](#)



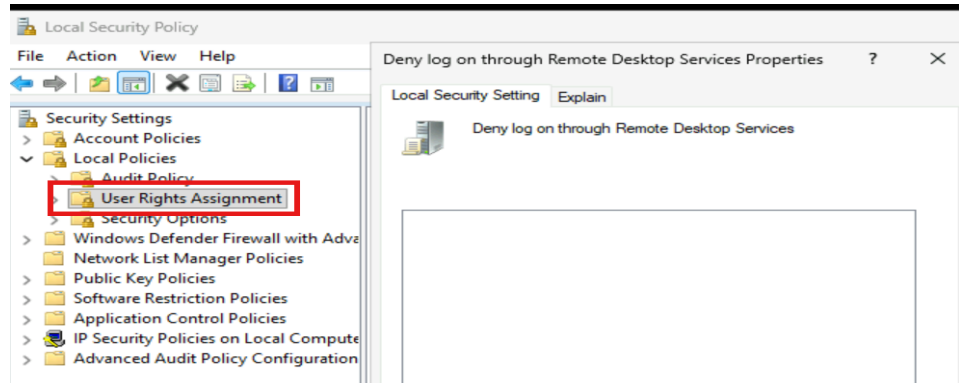
Phase 5 — Weaken & expose (deliberately, in order)

Only after detections are armed. Do these in sequence and record the exposure timestamp — it marks the start of the incident window.

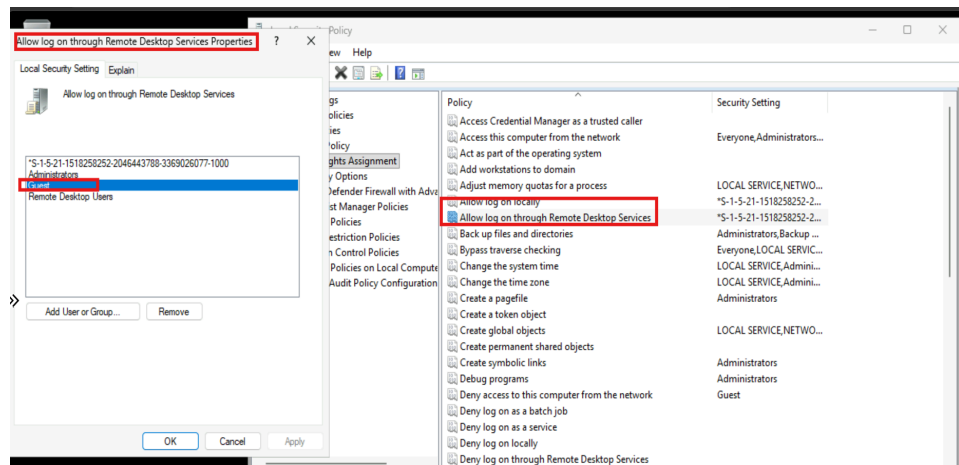
- ☐ Via compmgmt.msc, enable (or create) the “Administrator” on your VM
 - ☐ Ensure it's in the Administrators group, and set a weak password such as “password” or [something from RockYou.txt top 10 weakest passwords](#)
- ☐ Via compmgmt.msc, enable the “Guest” account and set the password to be blank
 - ☐ Add the guest account to the “Users” group



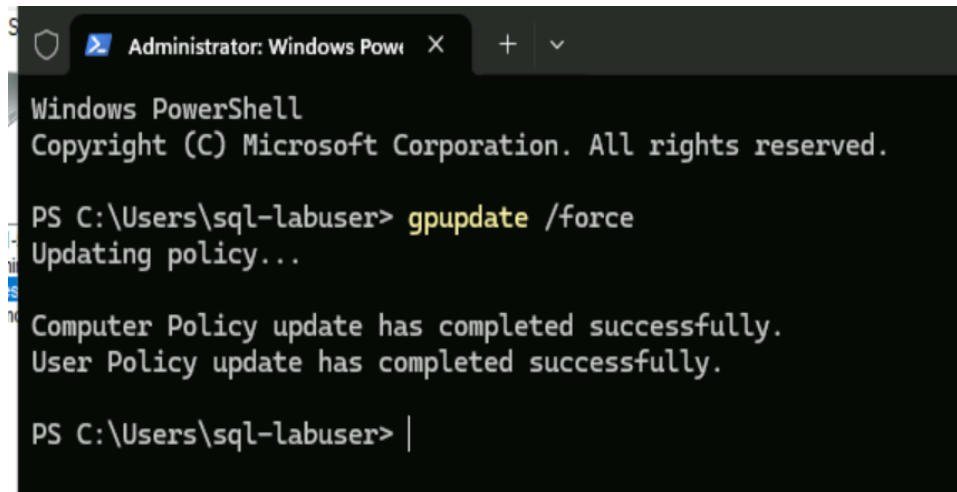
- ☐ Allow the guest account to login over the network: in secpol.msc → Local Policies → User Rights Assignment:
 - ☐ Deny log on through Remote Desktop Services — remove Guest from this list (it's there by default).



- ☐ Allow log on through Remote Desktop Services — make sure Guest (or Remote Desktop Users) is listed.



- ☐ Check Deny log on locally too, and remove Guest if present.
- ☐ in the same console → Security Options: Set Accounts: Limit local account use of blank passwords to console logon only → Disabled (or just give Guest a real password, which is cleaner).
- ☐ Under settings → Remote Desktop Users, add the Guest account
- ☐ Run “gpupdate /force”



```

Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\sql-labuser> gpupdate /force
Updating policy...

Computer Policy update has completed successfully.
User Policy update has completed successfully.

PS C:\Users\sql-labuser> |

```

- ☐ Enable MySQL authentication reachable over the network; set MySQL root user to use “root” as the password as well:

```

CREATE USER 'root'@'%' IDENTIFIED BY 'root';
GRANT ALL PRIVILEGES ON *.* TO 'root'@'%' WITH GRANT OPTION;
FLUSH PRIVILEGES;

```

Pre-Breach Exposure & Monitoring Checklist

Before intentionally exposing the VM, complete the following steps to establish a controlled baseline and ensure that all required telemetry and detections are operational.

- ☐ **Capture an Investigation Package** for the VM through Microsoft Defender.
Important: This package will serve as the **pre-breach baseline** for comparison during the post-breach investigation.
 - ☐ **Disable the Windows Firewall** within the authorized lab environment.
 - ☐ **Weaken the Network Security Group (NSG)** by allowing inbound traffic.
Lab Warning: This intentionally increases the VM's network exposure and discoverability. Perform this step only within the isolated, authorized lab environment.
- ☐ **Record the exact exposure timestamp:** {2026-08-31T08:06:09.2715604Z}
- **Investigation Note:** This timestamp establishes the beginning of the exposure window and will be used to calculate the elapsed time between exposure and the first observed compromise activity.
 - ☐ **Confirm that all required detections, logging, and monitoring controls are enabled and operational** before leaving the VM exposed.
 - ☐ **Confirm the VM's automatic shutdown schedule.**
The VM is configured to shut down automatically at midnight EST each night for **cost control and lab containment**. If the lab is still active, restart the VM the following morning and verify that monitoring and detection controls remain operational.

Investigation Objective

The purpose of this phase is to establish a clear **pre-breach baseline**, intentionally expose the lab system, and preserve the telemetry required to measure and investigate the subsequent attack lifecycle.

Exposure Timestamp → Initial Access → Authentication → Execution → Persistence → Post-Breach Investigation

VECTOR DESIGN

Recommended path is RDP-breach-then-pivot: the box is brute-forced over RDP (the real attacker traffic you already see), the attacker then discovers and accesses the local MySQL and its dummy data. That yields a richer chain — initial access → discovery → collection — than a flat “someone hit 3306.” Exposing 3306 directly as well is the insurance policy if you want to guarantee the DB gets touched within a student’s window.

Phase 6 — Detect the Breach

Ensure your VM remains online and wait for your analytics rules to trigger.

Keep watching for logon activity against your VM and watching for incidents to be created in Defender/Sentinel based on your Analytics Rule. You can look through DeviceLogonEvents and MySQLAudit_CL, filtering for your device and looking at the latest logs.

For example, I might use the detection/analytics rule queries as helper queries to help me monitor activity

```
// SQL Server
let MyDevice = "sql-labuser";
let MyTimeframe = todatetime("2026-08-31T08:06:09.2715604Z");
let FailedConnections =
MySQLAudit_CL
| extend RawData = replace_string(RawData, "\t", " ")
| extend DeviceName = tostring(split(_ResourceId, "/")[-1])
| where DeviceName == MyDevice
| where RawData has "Access denied"
| extend ConnectionId = extract(@"^S\s+(\d+)\s+Connect", 1, RawData)
| distinct ConnectionId;
MySQLAudit_CL
//| where TimeGenerated > MyTimeframe
| extend RawData = replace_string(RawData, "\t", " ")
| extend DeviceName = tostring(split(_ResourceId, "/")[-1])
| where DeviceName == MyDevice
| where RawData has "Connect"
| extend ConnectionId = extract(@"^S\s+(\d+)\s+Connect", 1, RawData)
| extend ActionType =
    case(
        RawData has "Access denied", "LogonFailure",
        ConnectionId in (FailedConnections), "Ignore",
        "LogonSuccess"
```

```

)
| where ActionType != "Ignore"
| extend RawData = replace_string(RawData, "\t", " ")
| extend Username = 
replace_string(tostring(split(tostring(split(RawData,"@")[0]), " ")[-1]), "'", 
"")
| extend IPAddress = replace_string(tostring(split(split(RawData,"@")[1], " 
")[0]), "'", "")
| project TimeGenerated, DeviceName, Username, IPAddress, ActionType, RawData
| order by TimeGenerated desc

```

// Filtering Queries

```

let MyDevice = "sql-labuser"; // set your own device name
let ServerVulnerableDateTime = todatetime("2026-08-31T08:06:09.2715604Z");
MySQLAudit_CL
| where TimeGenerated > ServerVulnerableDateTime
| where RawData has "Query"
| extend RawData = replace_string(RawData, "\t", " ")
| extend DeviceName = tostring(split(_ResourceId, "/")[-1])
| where DeviceName == MyDevice
| extend ActionType = "Query"
| extend Query = split(RawData, "Query")[1]
| project TimeGenerated, DeviceName, ActionType, Query, RawData
| order by TimeGenerated desc

```

// Virtual Machine Logons

```

let MyDevice = "sql-labuser"; // set your own device name
let ServerVulnerableDateTime = todatetime("2026-08-31T08:06:09.2715604Z");
DeviceLogonEvents
| where TimeGenerated > ServerVulnerableDateTime
| where DeviceName == MyDevice
| where AccountName in~ ("administrator", "guest")
| project TimeGenerated, RemoteIP, AccountName, DeviceName, ActionType, LogonType

```

When a breach finally happens, continue to monitor what the attacker is doing through the other tables.

A breach on the VM? Use: DeviceProcessEvents, DeviceFileEvents, DeviceNetworkEvents, etc, filter for your device as well as the “administrator” account.

You can also observe the NTANetAnalytics table to observe any traffic to/from your VM after the breach:

```

let MyDevice = "sql-labuser";
NTANetAnalytics
| where isnotempty(SrcVm)
| where SrcVm endswith MyDevice
| where DeniedOutFlows >= 1

```

```
| project TimeGenerated, DeviceName = MyDevice, FlowType, FlowStatus, SrcIp,
SrcPorts, DestIp, DestPort
```

A breach on MySQL? Use: MySQLAudit_CL to filter for commands, consider the following query:

```
// Filtering Queries
MySQLAudit_CL
| where RawData has "Query"
| extend RawData = replace_string(RawData, "\t", " ")
| extend DeviceName = tostring(split(_ResourceId, "/")[-1])
| extend ActionType = "Query"
| extend Query = split(RawData, "Query")[1]
| project TimeGenerated, DeviceName, ActionType, Query, RawData
| order by TimeGenerated desc
```

Phase 7 — Analyze the Breach

From here on out, we can't do a normal step-by-step instruction because what ends up happening to your environment is purely dynamic and will depend on when/how it gets breached.

After a day or so, assuming your VM/MySQL database is reachable, you should start getting indicators of compromise or signs of people poking around, trying to get in.

Open a new google doc to take notes / keep track of queries: [Queries and AI Analysis](#) (make a copy of this)

Continue VM & MySQL Authentication Analysis

Continue analyzing **VM and MySQL authentication activity** using the KQL queries provided in the previous sections.

If an unauthorized user successfully authenticates to **MySQL**, continue monitoring the MySQL query logs to determine what commands and database operations are being performed.

If an unauthorized user successfully authenticates to the **virtual machine** using either the **Guest** or **Administrator** account, investigate the endpoint activity that follows the successful logon.

The following Microsoft Defender for Endpoint tables should be reviewed:

1. DeviceLogonEvents — Virtual Machine Logons

Identify successful logons using the **Administrator** or **Guest** accounts after the vulnerable period.

```
// Virtual Machine Logons
```

```
let MyDevice = "sql-labuser";
let ServerVulnerableDateTime = todatetime("2026-08-31T08:06:09.2715604Z");
```

```
DeviceLogonEvents
| where Timestamp > ServerVulnerableDateTime
| where DeviceName =~ MyDevice
| where AccountName in~ ("administrator", "guest")
| where ActionType == "LogonSuccess"
| project Timestamp, RemoteIP, AccountName, DeviceName, ActionType, LogonType
| order by Timestamp asc
```

Investigation focus:

- Successful Administrator or Guest logons
- Source/remote IP addresses
- Logon type
- Authentication timing
- Repeated or unusual authentication activity

2. DeviceProcessEvents — Process Execution

Review processes executed after the **first successful login** to identify commands, scripts, or applications launched during the authenticated session.

```
let FirstSuccessfulLoginTime = todatetime("2026-08-31T15:26:56.2291226Z");
let MyDevice = "sql-labuser";
DeviceProcessEvents
| where DeviceName =~ MyDevice
// | where AccountName =~ "Administrator"
| where Timestamp >= FirstSuccessfulLoginTime
| project Timestamp, AccountName, DeviceName, ProcessCommandLine,
InitiatingProcessCommandLine,
    FolderPath, FileName
| order by Timestamp asc
```

Investigation focus:

- Processes launched immediately after authentication
- Suspicious command-line activity
- PowerShell or scripting activity
- Unexpected administrative utilities

- Parent/child process relationships
- Execution from unusual directories

3. DeviceFileEvents — File Activity

Review file creation, modification, and other file-system activity occurring after the first successful login.

```
let FirstSuccessfulLoginTime = todatetime("2026-08-31T15:26:56.2291226Z");
let MyDevice = "sql-labuser";
DeviceFileEvents
| where DeviceName =~ MyDevice
| where Timestamp >= FirstSuccessfulLoginTime
// Uncomment to focus on activity performed by a specific account
// | where InitiatingProcessAccountName =~ "Administrator"
| project Timestamp, DeviceName, ActionType, FileName, FolderPath, SHA1, SHA256,
    InitiatingProcessAccountName, InitiatingProcessFileName, InitiatingProcessCommandLine,
    InitiatingProcessFolderPath, InitiatingProcessParentFileName, InitiatingProcessParentId
| order by Timestamp asc
```

Investigation focus:

- Newly created or modified files
- Executables and scripts
- Files written to suspicious directories
- File hashes for IOC enrichment
- Processes responsible for file activity
- Potential payload or tool deployment

4. DeviceRegistryEvents — Registry Activity

Review registry modifications that occurred after the first successful login. This can help identify configuration changes or potential persistence mechanisms.

```
let FirstSuccessfulLoginTime = todatetime("2026-08-31T15:26:56.2291226Z");
let MyDevice = "sql-labuser";
DeviceRegistryEvents
| where DeviceName =~ MyDevice
| where Timestamp >= FirstSuccessfulLoginTime
// Uncomment to focus on activity performed by a specific account
```

```
// | where InitiatingProcessAccountName =~ "Administrator"
| project
    Timestamp,
    DeviceName,
    ActionType,
    RegistryKey,
    RegistryValueName,
    RegistryValueData,
    InitiatingProcessAccountName,
    InitiatingProcessFileName,
    InitiatingProcessCommandLine,
    InitiatingProcessFolderPath,
    InitiatingProcessParentFileName,
    InitiatingProcessParentId,
    InitiatingProcessSHA1,
    InitiatingProcessSHA256
| order by Timestamp asc
```

Investigation focus:

- Registry modifications following authentication
- Run/RunOnce persistence locations
- New or modified registry values
- Processes responsible for registry changes
- Suspicious command lines
- File and registry activity occurring within the same timeframe

Correlation Objective

The goal of this phase is to correlate **authentication** → **process execution** → **file activity** → **registry changes**.

A successful login by itself does not necessarily indicate compromise. However, if the authentication event is followed by suspicious process execution, unexpected file creation, or registry modifications, those events should be correlated and incorporated into the investigation timeline.

Investigation Flow:

Successful Logon → **Process Execution** → **File Activity** → **Registry Changes** → **Persistence / Further Activity**

Continue correlating these findings with **MySQL authentication and query activity** to determine whether database access and host-level activity are related to the same source, account, or timeline.

- Or even DeviceNetworkEvents and NTANetAnalytics

Let the bad actor(s) sit for AT LEAST 24 hours or until you are satisfied with the data you have collected.

My VM and Database were online for at least 48 hours before the following happened and I decided to shut everything down.

Analyze the logs with ChatGPT:

MySQL Server Authentication Prompt:

You are a god-tier cybersecurity analyst with world-class threat hunting skills. Please look at these logs, these are MySQL Server Authentication logs from my environment. Please analyze them and paint a picture for what might be going on.

Here's the full threat picture — this log tells a clear, multi-stage story.

MySQL Authentication Log Analysis — **sql-labuser**

Timeframe: 2026-09-01 20:27 UTC → 2026-09-02 07:47 UTC (~11.5 hours)

What Happened — The Narrative

Phase 1 — External Brute Force / Credential Spray (Sep 1, 20:27–21:02 UTC)

Three distinct external IPs — all within the same /24 subnet (64.89.163.x) — attempted to authenticate as root over TCP/IP. The pattern is textbook: each session begins with a failed root login using no password (using password: NO), immediately followed by one or more successes. This failure-then-success pattern strongly suggests an **automated tool iterating through credentials**, where the first attempt is a blank password probe and subsequent attempts carry the actual credential.

IP	Failures	Successes	Window
64.89.163.166	2	4	20:27–20:28 UTC

64.89.163.1 45	1	0	21:02 UTC
64.89.163.1 39	2	4	04:16 UTC (Sep 2)

The fact that all three IPs sit in the same `/24` block (`64.89.163.0/24`) is significant — this is almost certainly **the same actor rotating IPs**, likely through a VPS pool or proxy infrastructure, to evade per-IP rate limiting or firewall rules.

Phase 2 — Lateral Pivot to Localhost (Sep 2, 04:26 UTC)

Approximately 10 minutes after the last external IP session (`64.89.163.139`), `root` authentication activity moves to `localhost` — and critically, it arrives **over SSL/TLS**, not TCP/IP. This suggests something or someone is now operating **on the box itself**. The same failure-then-success pattern repeats (`using password: NO` fails, then two successes follow seconds later), but from `localhost`, implying a local process or shell was used to probe MySQL directly.

This is consistent with **post-exploitation lateral movement**: the attacker gained a foothold on the host (perhaps via OS-level access) and is now authenticating to MySQL locally to enumerate or exfiltrate data.

Phase 3 — Anonymous/Blank Connections (Sep 2, 06:23–07:47 UTC)

Three successful `LogonSuccess` events with **no username and no IP address** recorded. This is the most anomalous part of the entire log. In MySQL's audit log, blank username and IP can occur in two scenarios:

1. **Anonymous user account** — MySQL's `'@'` or `'@localhost'` anonymous user, a legacy feature that should not exist in a hardened environment. If enabled, it allows unauthenticated access.
2. **Audit plugin gap / evasion** — Certain MySQL connection methods or plugin configurations can result in incomplete audit records. A sophisticated actor aware of MySQL internals could exploit this.

These connections succeed (conn IDs 21, 22, 23), span nearly 90 minutes, and follow the localhost pivot — the timing is not coincidental. These look like **post-compromise sessions operating under an anonymous or stripped identity to reduce forensic visibility**.

Key Indicators of Compromise (IoCs)

IoC Type	Value
----------	-------

External IPs	64.89.163.139, 64.89.163.145, 64.89.163.166
IP Subnet	64.89.163.0/24
Target Account	root (MySQL)
Device	sql-labuser
Auth Pattern	password: NO failure → immediate success
Anomaly	3x anonymous successful connections (no user/IP)

Priority Investigative Actions

Immediate pivots from here:

- **OSINT the /24 block** — Run 64.89.163.0/24 through Shodan, AbuseIPDB, and WHOIS. Identify if this is a known VPS/hosting provider used by threat actors.
 - **Check MySQL general query log** — Authentication success tells you they got in; the general log tells you what they *did*. Look for **SELECT**, **INTO OUTFILE**, **LOAD DATA**, **SHOW DATABASES** after each successful connection.
 - **Investigate the anonymous user** — Run **SELECT user, host, authentication_string FROM mysql.user WHERE user=''** on the server. If an anonymous account exists, it needs to be dropped immediately.
 - **Check OS logs around 04:16–04:26 UTC** — The pivot to localhost means something executed on the host. Correlate with SSH auth logs, process creation events, cron jobs, or web server logs from that window.
 - **Look for conn IDs 19–20** — The connection ID jumps from 18 to 21. Something happened in that gap that wasn't captured (or was deleted) from this audit log.
 - **Examine SSL certificate used** — The localhost sessions used SSL/TLS. Identify what cert was presented and whether it matches expected service credentials.
-

Bottom Line Assessment

This log captures what appears to be a **successful external brute-force followed by host compromise and post-exploitation MySQL access**. The attacker rotated IPs within a /24, cracked **root** credentials, pivoted to localhost (suggesting OS-level access was obtained), and then established anonymous MySQL sessions — likely to operate with reduced audit trail. The 11.5-hour window and methodical progression suggest an **automated initial compromise followed by hands-on post-exploitation activity**.

Treat **sql-labuser** as fully compromised until proven otherwise.

MySQL Server Query Prompt:

You are a god-tier cybersecurity analyst with world-class threat hunting skills.
 Please look at these logs, these are MySQL Server Query logs from my environment.
 Please analyze them and paint a picture for what might be going on.

This is a devastating log. Here is the complete picture — this is a confirmed, fully executed ransomware attack.

MySQL Query Log Analysis — `sql-labuser`

Full Timeframe: 2026-08-31 08:09 UTC → 2026-09-02 04:26 UTC

What Happened — The Complete Attack Narrative**Phase 0 — Pre-Compromise Reconnaissance (Aug 31, 08:09 UTC)**

The earliest activity shows two simultaneous connections (conn 9 and conn 10) running classic **MySQL Workbench initialization fingerprints** — `show engines`, `show charset`, `show collation`, `SHOW SESSION VARIABLES LIKE 'version_comment'`, `show variables`. This is an operator using MySQL Workbench to explore an environment they've just accessed. They navigate to `sakila.address` and then `lnp_corp.payments`, running `SELECT *` queries against a corporate payments table — **the first data access of the breach**. They hit `lnp_corp.payments` repeatedly over a span of four minutes, suggesting they were confirming data was present and reviewing what they had.

Phase 1 — Persistence Implant (Sep 1, 00:08 UTC — ~16 hours later)

The actor returns. This time, after reconnecting and running the same Workbench fingerprinting queries, they execute three decisive DDL statements in rapid succession:

```
CREATE USER 'root'@'%' IDENTIFIED BY <secret>
GRANT ALL PRIVILEGES ON *.* TO 'root'@'%' WITH GRANT OPTION
FLUSH PRIVILEGES
```

This is **backdoor account creation**. The `'%'` wildcard means this `root` account can connect from **any IP address** with full superuser privileges. This is their persistence mechanism — they now own the server permanently, regardless of what happens to the original compromised credential.

Phase 2 — Ransomware Tool Deployment and Full Database Exfiltration (Sep 1, 04:57 UTC)

Approximately five hours after planting the backdoor, the actor deploys their automated ransomware tool. The tool signature is unmistakable — the MariaDB/MySQL dump utility (`mysqldump` with MariaDB comment syntax `/*M!100100 ...*/`), spawning dozens of parallel connections in the conn 108–190 range.

The tool methodically targets every database, performing for each table: schema inspection via `INFORMATION_SCHEMA`, column enumeration, then `SELECT ... WHERE 1 LIMIT 10` sample reads (confirming data exists) followed by a full `SELECT *` dump. The exfiltration order reveals exactly what was stolen:

From **lnp_corp** (corporate database — highest severity):

Table	Columns Confirmed Exfiltrated
<code>credential</code> <code>ls</code>	<code>cred_id</code> , <code>customer_id</code> , <code>username</code> , <code>password</code> , <code>role</code> , <code>last_login</code>
<code>customers</code>	<code>first_name</code> , <code>last_name</code> , <code>email</code> , <code>phone</code> , <code>street</code> , <code>city</code> , <code>state</code> , <code>zip</code> , <code>date_of_birth</code> , <code>ssn</code>
<code>orders</code>	<code>order_id</code> , <code>customer_id</code> , <code>order_date</code> , <code>amount</code> , <code>status</code>
<code>payments</code>	<code>payment_id</code> , <code>order_id</code> , <code>card_brand</code> , <code>card_masked</code> , <code>card_last4</code> , <code>exp_month</code> , <code>exp_year</code>

This is a **complete PII and financial data breach** — full names, SSNs, DOBs, card data, and plaintext/hashed credential pairs.

From **sakila** and **world**: All tables in both databases were enumerated and dumped (actor, address, city, country, film, inventory, payment, rental, staff, store, and more).

Phase 3 — Destructive Execution (Sep 1, 04:57–04:58 UTC)

Immediately after confirming exfiltration of **lnp_corp**, the tool executes a clean, methodical destruction sequence. The pattern for each database is identical:

1. **Map all foreign keys** via `information_schema.KEY_COLUMN_USAGE`
2. **Drop all foreign key constraints** (`ALTER TABLE ... DROP FOREIGN KEY`)
3. **Drop all tables** in dependency order
4. **Create ransom table**: `CREATE TABLE RECOVER_YOUR_DATA`
5. **Insert ransom note** and **COMMIT**

This was executed across three databases in sequence — **lnp_corp** → **sakila** → **world** — all within approximately 90 seconds.

The ransom note left in every destroyed database:

*"All your data was backed up by us. You must pay 0.0107 bitcoin to [bc1q017hr5v220f5q1qhjhq4p3jjqfkgudazzjn1kw](#) or in 48 hours, your data will be publicly disclosed and deleted. After payment send mail to [ak+2muz2@onionmail.org](#)... Your DATAID is: **2MUZ2**"*

Phase 4 — Anti-Forensics and Cover Tracks (Sep 1, 04:58 UTC)

After destruction, the actor executes a deliberate anti-forensics sequence within seconds:

```
RESET MASTER                -- wipes binary log files
PURGE BINARY LOGS TO 'josh-mde-lab-bin.000001' -- purges remaining binlogs
REVOKE INSERT, UPDATE, DELETE, DROP, CREATE ON *.* FROM 'root'@'%' -- modifies
backdoor permissions
SHUTDOWN                    -- kills the MySQL process
```

RESET MASTER is the most telling query in the entire log. It is the canonical MySQL anti-forensics command — it destroys the binary log, which is normally the gold-standard recovery and forensic evidence source. They then purged any remaining log files. This actor was specifically trying to prevent recovery and evidence collection.

Phase 5 — Repeat Visits (Sep 1–2, 20:28 and 04:16 UTC)

The external IPs from the authentication log ([.166](#) and [.139](#)) come back after the attack. In both sessions, after authenticating, the tool immediately runs **SHOW DATABASES** and then **CREATE DATABASE IF NOT EXISTS RECOVER_YOUR_DATA**. This is the tool **checking whether the victim paid** and re-establishing the ransom message if the database was restored. Standard ransomware operator behavior.

Phase 6 — Human Operator Returns (Sep 2, 04:26 UTC)

The final session from [localhost](#) is a **human returning via MySQL Workbench** (same Workbench fingerprint queries as Day 1). They then execute:

```
CREATE USER 'labadmin'@'%' IDENTIFIED BY <secret>
GRANT ALL PRIVILEGES ON *.*
FLUSH PRIVILEGES
CREATE USER 'root'@'%' IDENTIFIED BY <secret> -- re-implanting the backdoor
```

The backdoor was re-established. The SHUTDOWN command may have wiped the earlier user, so they are re-planting persistence. This is hands-on operator activity post-automation.

Attack Timeline Summary

Time (UTC)	Event
Aug 31 08:09	Initial access, Workbench reconnaissance, <code>lnp_corp.payments</code> accessed
Sep 1 00:08	Return visit — backdoor account <code>root@%</code> created with full privileges
Sep 1 02:59	Connection 108 — <code>SHOW DATABASES</code> (tool probe)
Sep 1 04:57	Ransomware tool launches — full schema/data exfiltration begins
Sep 1 04:57:35	<code>DROP TABLE payments/orders/customers/credentials</code> — <code>lnp_corp</code> destroyed
Sep 1 04:58:41	<code>sakila</code> destroyed
Sep 1 04:58:52	<code>world</code> destroyed
Sep 1 04:58:53	All three databases dropped at the schema level
Sep 1 04:58:55	<code>RESET MASTER</code> — binary logs wiped
Sep 1 04:58:57	<code>SHUTDOWN</code> — server killed
Sep 1 20:28	IP <code>.166</code> returns — checks for payment, re-inserts ransom note
Sep 2 04:16	IP <code>.139</code> returns — same check
Sep 2 04:26	Human operator returns via localhost/Workbench — re-plants backdoor accounts

Critical Findings

Confirmed Data Exfiltrated:

- Full customer PII from `lnp_corp.customers` including **SSNs and dates of birth**
- Plaintext or hashed **credential pairs** (username + password) from `lnp_corp.credentials`
- **Payment card data** (brand, masked number, last 4, expiry)
- All financial transactions (`orders`, `payments`)
- Complete `sakila` and `world` datasets

Threat Actor Characteristics:

- Highly automated toolchain (mysqldump-based, parallel connections)

- Manual operator conducting reconnaissance and persistence (MySQL Workbench fingerprint)
- Anti-forensics aware (**RESET MASTER**, binlog purge, **SHUTDOWN**)
- Known ransomware campaign — Bitcoin wallet **bc1q017hr5v220f5qlqhjh4p3jjqfkgudazzjnlkw** and **onionmail.org** contact should be threat-intel'd immediately
- DATAID **2MUZZ** is a victim tracking identifier — this is a ransomware-as-a-service (RaaS) or repeat-campaign operator

Immediate Actions Required:

- **Do not pay the ransom** — evaluate actual backup posture independent of the attacker's claim to hold your data
- Revoke and rotate all MySQL accounts, particularly **root@%** and **labadmin@%**
- Block **64.89.163.0/24** at the network perimeter immediately
- Notify legal and privacy counsel — SSN and payment data exposure likely triggers breach notification obligations (depending on jurisdiction)
- Search threat intel platforms for the Bitcoin address, email, and DATAID **2MUZZ** to identify campaign scope
- Rebuild **sql-labuser** from scratch — treat the entire host as unrecoverable

Defender Logs Prompt:

You are a god-tier cybersecurity analyst with world-class threat hunting skills. Please look at these logs, these are Microsoft Defender (MDE) logs from my environment. Please analyze them and paint a picture for what might be going on.

- DeviceLogonEvents
- DeviceProcessEvents
- DeviceFileEvents
- DeviceRegistryEvents
- Or even DeviceNetworkEvents and NTANetAnalytics
-



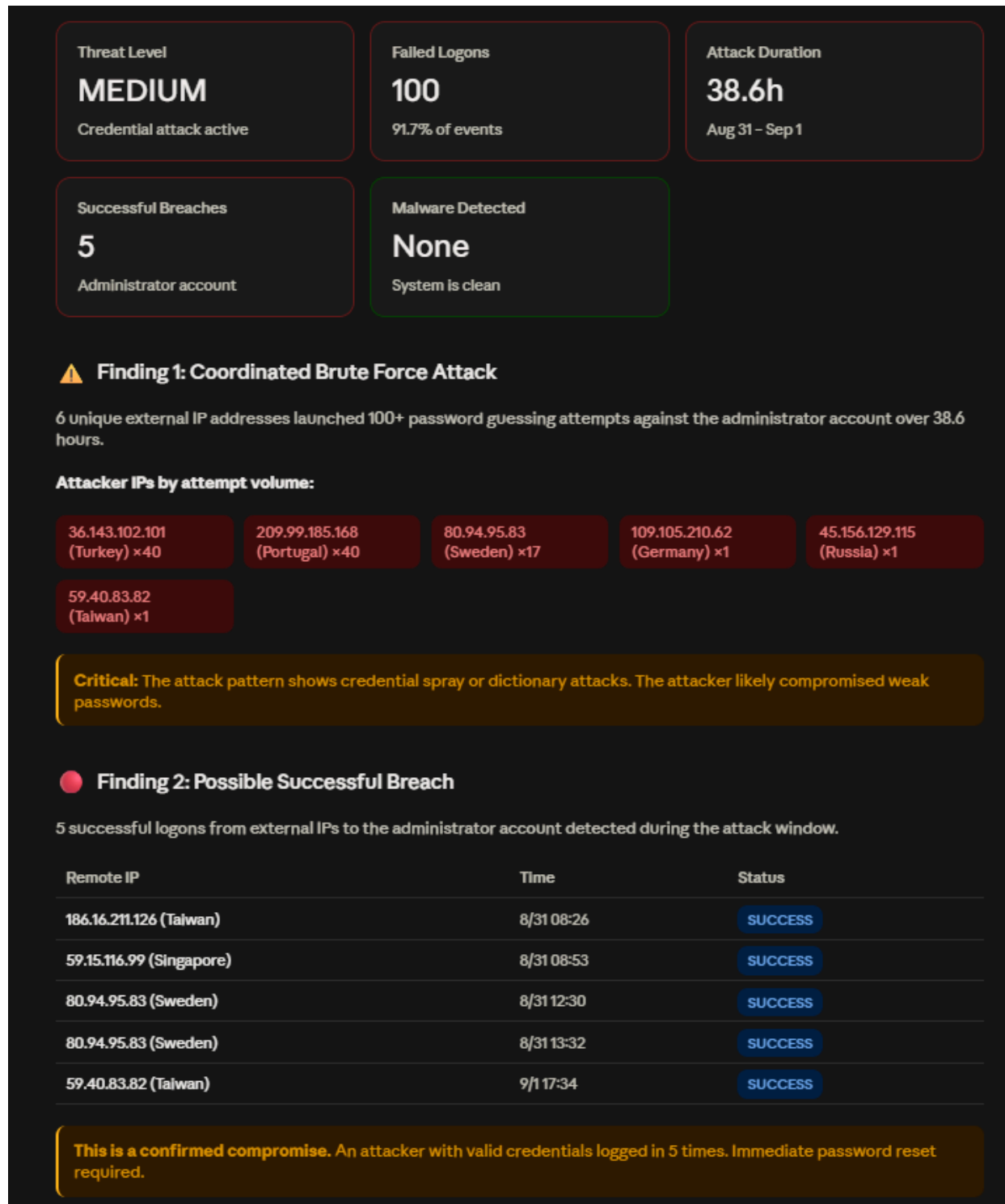
Threat Hunting Report: sql-labuser

Microsoft Defender Advanced Threat Protection (MDE) Analysis

The Attack Narrative

Your environment experienced a **coordinated credential attack** starting at 03:12 UTC on August 31st. Six compromised or bot-controlled external IPs began a systematic brute force campaign against your **administrator** account, trying common passwords and variations. The attackers maintained pressure for 38+ hours with varying intensity — two IPs (36.143.102.101 from Turkey and 209.99.185.168 from Portugal) each made 40 attempts, suggesting they're primary nodes in the attack network.

The attack worked. At 08:26 on August 31st, someone from Taiwan (186.16.211.126) successfully authenticated as administrator. By that evening, two more distinct foreign IPs had gained access. Over the next 24 hours, the same Swedish IP (80.94.95.83) logged in a second time, and on September 1st, yet another Taiwanese IP (59.40.83.82) obtained access.



Finding 3: No Malware or Persistence Detected

All system activity is consistent with legitimate Windows and Azure operations.

- ☐ No malware/trojans
- ☐ No ransomware signatures
- ☐ No persistence mechanisms
- ☐ No lateral movement detected
- ☐ No privilege escalation
- ☐ No data exfiltration indicators

All file/registry changes legitimate (MDE telemetry, Azure Guest Config, Windows Update)

Immediate Actions Required

- ☒ Reset administrator password NOW – Choose a 20+ character random password
- ☒ Enable MFA – Add multi-factor authentication to all admin accounts immediately
- ☒ Block attacker IPs – Add the 6 IPs above to your firewall/WAF block list
- ☒ Review successful sessions – Check Event Logs for actions taken 8:26–9/1 17:34 under administrator account
- ☒ Check for lateral movement – Search for admin RDP/SSH connections to other hosts (unlikely but verify)
- ☒ Audit recent changes – Review file uploads, script execution, scheduled tasks, user account changes
- ☒ Enable account lockout policy – Fail-close on 5 wrong passwords to prevent future brute force
- ☒ Consider forensic imaging – If this is a production system, preserve evidence and escalate to incident response

What This Means

This is not a "might be a problem" scenario — **your administrator account credentials have been compromised**. An attacker has valid credentials and has demonstrated they can log in from outside your network. The log entries show they logged in **5 separate times**, meaning they're actively using the account or testing access.

The good news: they haven't deployed malware, created persistence mechanisms, or moved laterally to other systems (at least not yet, based on MDE visibility). This window between breach and escalation is your opportunity.

System Health Assessment

The bulk of your logs are legitimate activity:

- **MDE SenseIR** threat detection telemetry (constant PowerShell/CSC.exe compilation for threat signatures)
- **Azure Guest Configuration** compliance audits (DSC policy checks)
- **Windows Update** operations (AM_Delta_Patch, MpSigStub)

- **Normal Windows services** (conhost, net1, svchost, Edge browser)

No hidden code, no scheduled tasks created, no suspicious network connections in the logs. The system itself is clean, but the keys to the front door have been handed over.

Geopolitical Notes

The IPs span Asia-Pacific (Singapore, Taiwan, Hong Kong region) and Eastern Europe (Turkey, Portugal masking, Russia). This is typical of:

- Bulletproof hosting providers in permissive jurisdictions
- Rented botnet nodes
- VPN/proxy chains with poor accountability

Don't assume "where the IP is" = "where the attacker is" — they're covering their tracks.

Your Next 2 Hours

1. **Isolate and reset** — Change the administrator password to something strong (30+ chars, random, cryptographically generated). Do this from a trusted device or through a direct console connection if possible.
2. **MFA everywhere** — Even a basic TOTP or Microsoft Authenticator app dramatically raises the attacker's cost and makes the account unusable.
3. **Block IPs** — Firewall-level ingress block for the 6 IPs. Coordinate with your network ops/security team to push this to your perimeter immediately.
4. **Log review** — Pull Windows Event Log 4624 (successful logons) and 4688 (process creation) for the 08:26–17:34 window. What did the attacker do during those 5 sessions? Did they enable persistence? Create new accounts? Access sensitive files?

Your Next 24 Hours

5. **Behavioral analysis** — Cross-reference successful admin logon times with file access, process creation, and network traffic logs. Did the attacker dump the SAM? Exfil application databases? Modify code?
6. **Account audit** — List all local and domain user accounts, especially any created in the last 48 hours. Disable anything suspicious.
7. **Compliance notification** — Depending on your industry (finance, healthcare, critical infrastructure), you may have breach notification requirements. Start that process now.
8. **Implement controls** — Lock down the administrator role: disable remote logons outside a VPN, use a dedicated jump host, require certificate-based auth, implement privileged access workstation (PAW) practices.

Red Flags You Should Have Seen

The attack signatures are elementary:

- Network-sourced logons from geographically diverse, hostile jurisdictions
- 100+ failed attempts in rapid succession (should trigger alert thresholds)
- Successful logons following brute force (highly suspicious pattern)

If you weren't alerted to this, your monitoring either isn't running, isn't configured, or the alerts are lost in noise. That's a second-order problem to solve after containment.

Bottom line: You've got a compromised administrator account, no malware yet, and a narrow window before the attacker escalates. Act now.

When you have analyzed all of the logs and dumped your findings and note into your copy of [Helper Queries and AI Analysis](#), you can move on to Phase 8

Phase 8 — Contain the Breach (Isolation)

Ensure the VM is powered on. From the Microsoft Defender portal, **isolate the device** and then capture a new **Investigation Package**. This post-isolation package will be compared with the initial collection to identify system changes, suspicious activity, and additional indicators of compromise.

Capture the exact time of isolation: 2026-09-02T19:34:07.0191605

- **Pre-breach collection : Abdifatah- Galgalo**
- **Post-breach collection -Abdifatah galgalo**
- **Post-Breach Isolation-adam**

 Verify the Isolation Timestamp

Use the following KQL query to display the current UTC time alongside the recorded device isolation timestamp:

```
let IsolationTime = datetime(2026-09-02T19:34:07.0191605Z);
```

```
print
```

```
    CurrentTime = now(),
```

```
    IsolationTime = IsolationTime
```

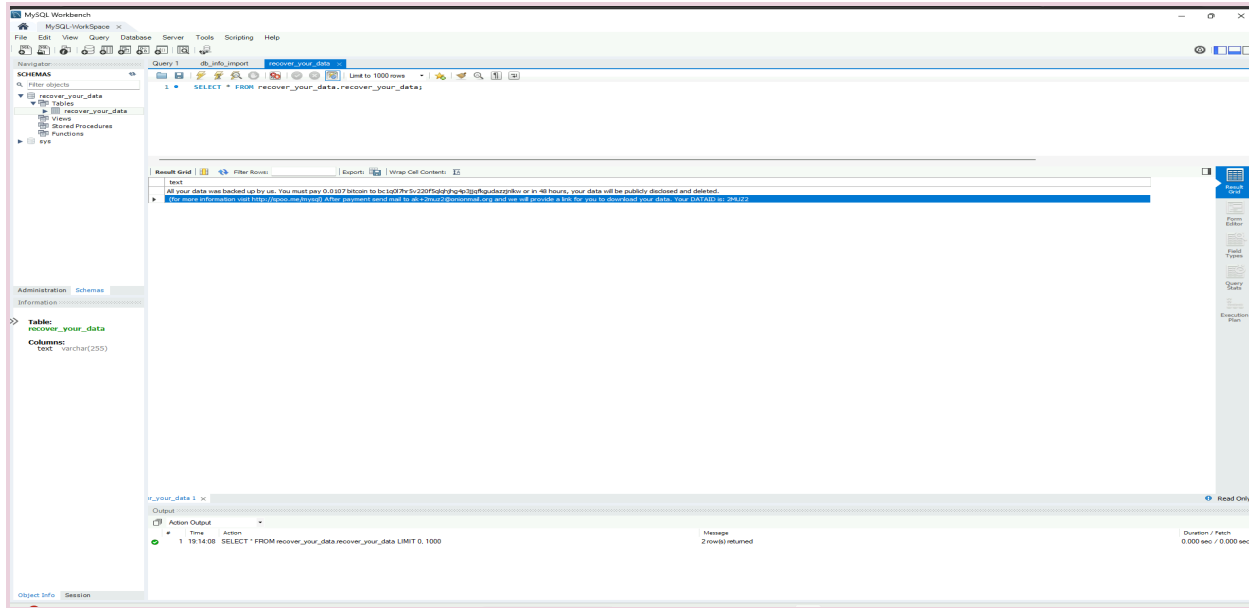
Why this matters: The exact isolation timestamp serves as a critical investigation boundary. It helps distinguish pre-isolation attacker activity from events that occurred after containment, making timeline reconstruction and package comparison more accurate.

SQL Injection — Ransom Message Example

The following is an example of a message that may be encountered after a **successful SQL injection attack**. In a real-world incident, an attacker may leave a ransom note or other instructions indicating that the database or sensitive information has been compromised.

The message may include **payment instructions, cryptocurrency wallet information, contact details, or links provided by the attacker**. Such content should be treated as a potential indicator of compromise and preserved as evidence during the investigation.

DFIR Note: Do not interact with attacker-provided links or contact information. Preserve the message, record the timestamp and affected system, and correlate it with database audit logs, authentication activity, process execution, and network telemetry.



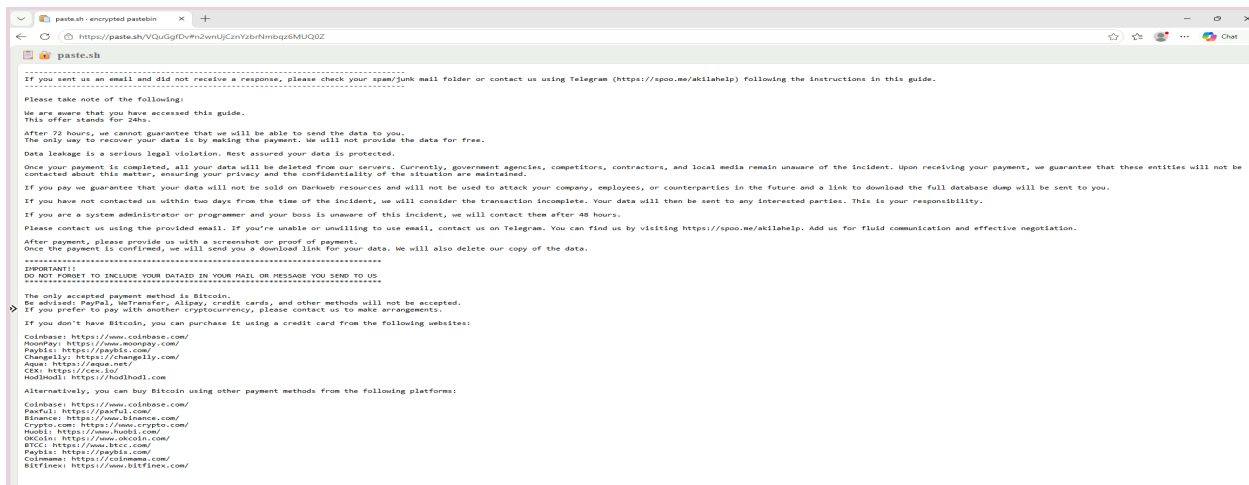
⚠ Lab Safety & Authorization Notice

The following scenarios are based on **real-world attack techniques observed in attacks targeting SQL servers**. These techniques are presented for **educational, defensive security, and incident-response purposes only**.

Do not attempt to reproduce or execute these techniques against personal, production, or unauthorized systems. All testing should be conducted exclusively within a **properly isolated, controlled, and explicitly authorized lab environment**.

Ensure that appropriate safeguards are in place before conducting any testing, including network isolation, disposable test systems, and controlled access to simulated data.

Authorization Requirement: Only perform security testing on systems and environments for which you have explicit permission to conduct the activity.

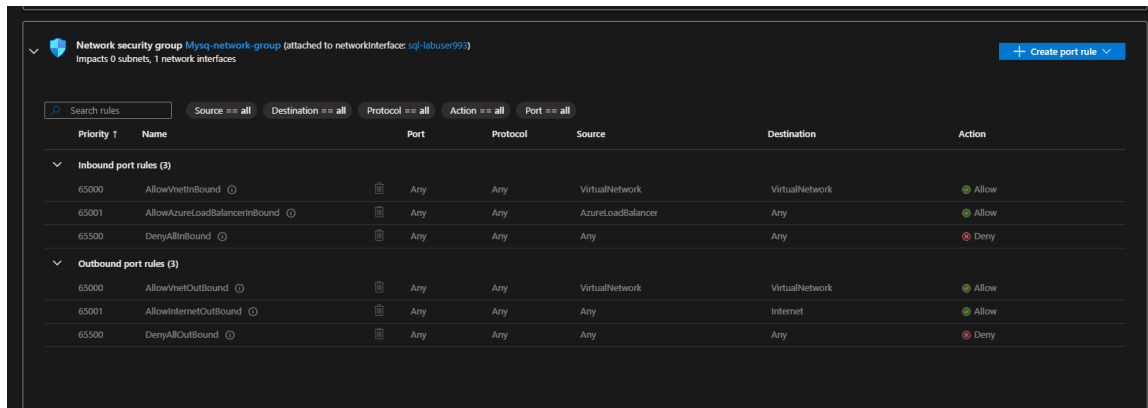


Phase 9 — Eradication and Recovery

What this phase looks like depends on what happened in your environment and what was on the system that got compromised. Realistically for this, since both the VM and the MySQL Server got compromised, the best thing to do would be to simply destroy the VM and restore the database from backup.

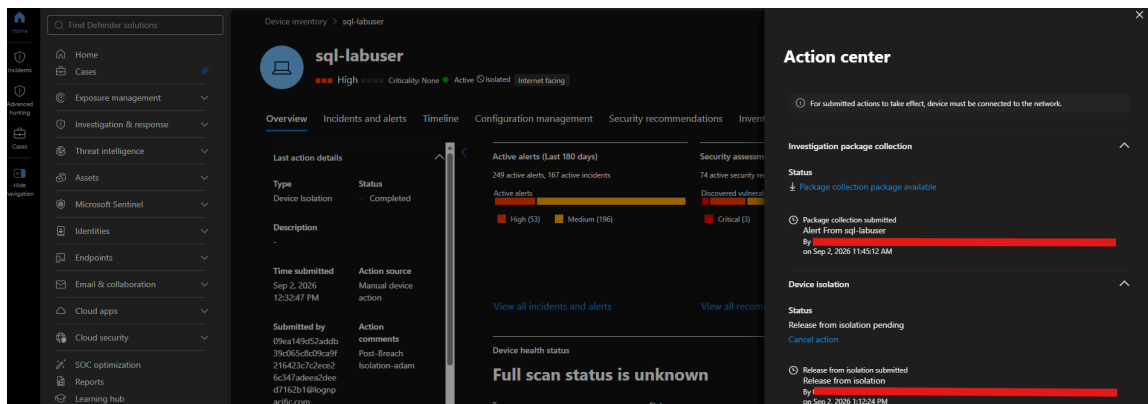
Alternatively, if you didn't want to (or couldn't) destroy the VM, you could harden the system, harden the database, then restore the MySQL data from backup with the following steps:

- ☐ Harden the VM's NSG by removing all the networking connection that's not harden

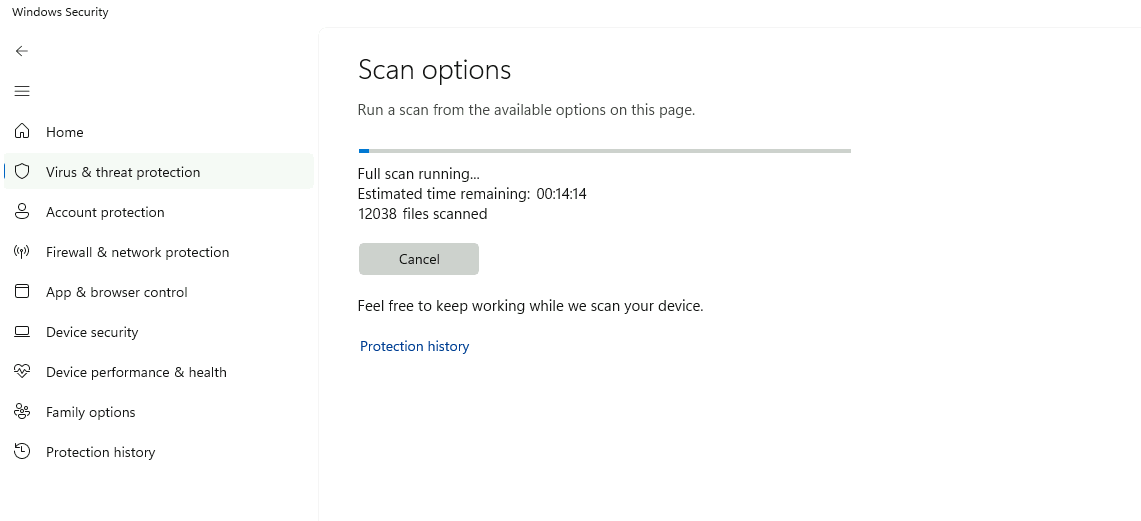


- ☐ Enable the Windows Firewall and perform a full firewall scan to ensure the system is properly protected.

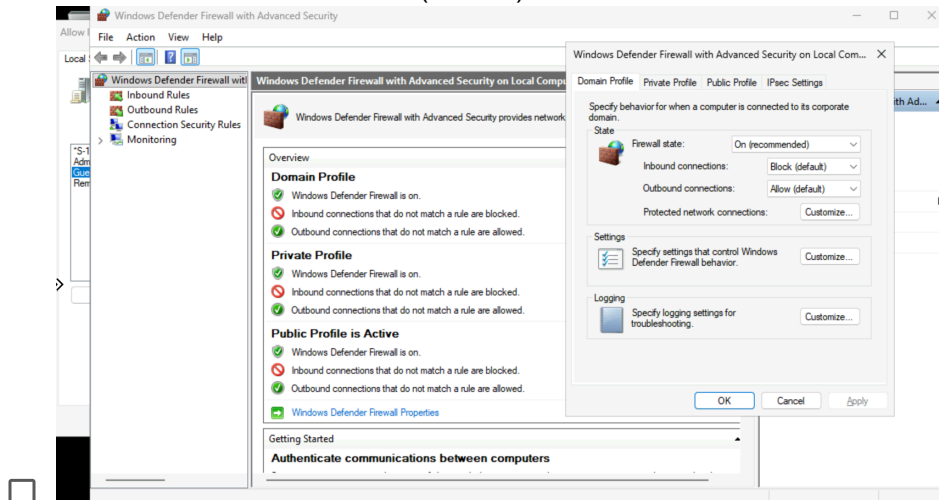
- ☐ Turn on VM
- ☐ Remove VM from Isolation



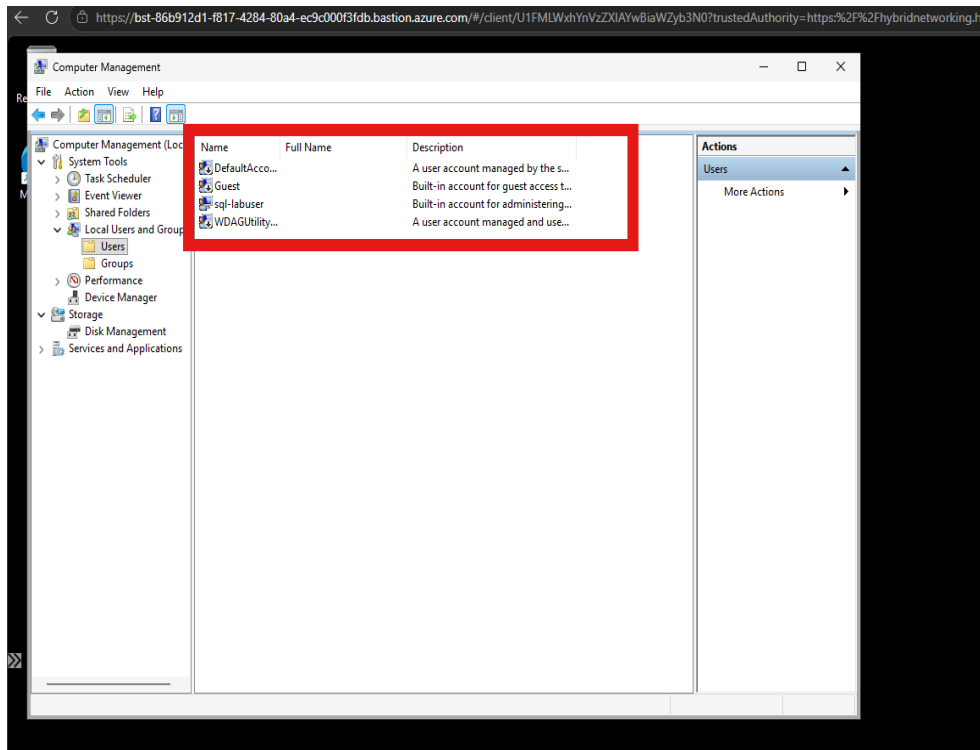
- ☐ Run a full malware scan using Windows Defender



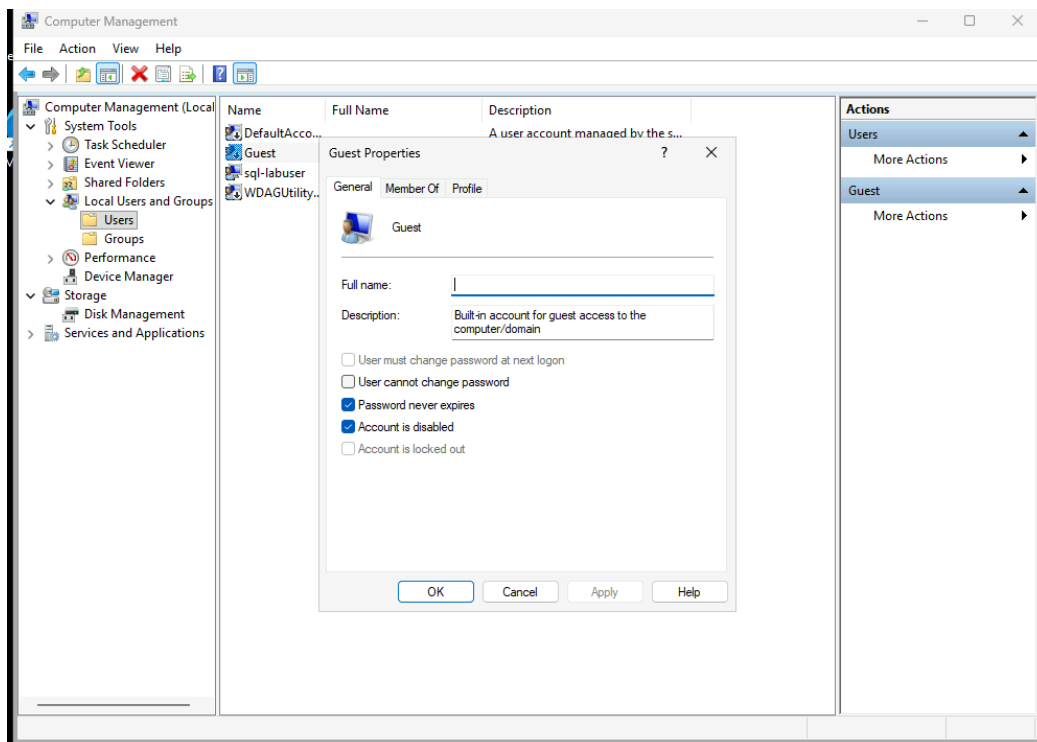
☐ Enable the Windows Firewall (wf.msc)



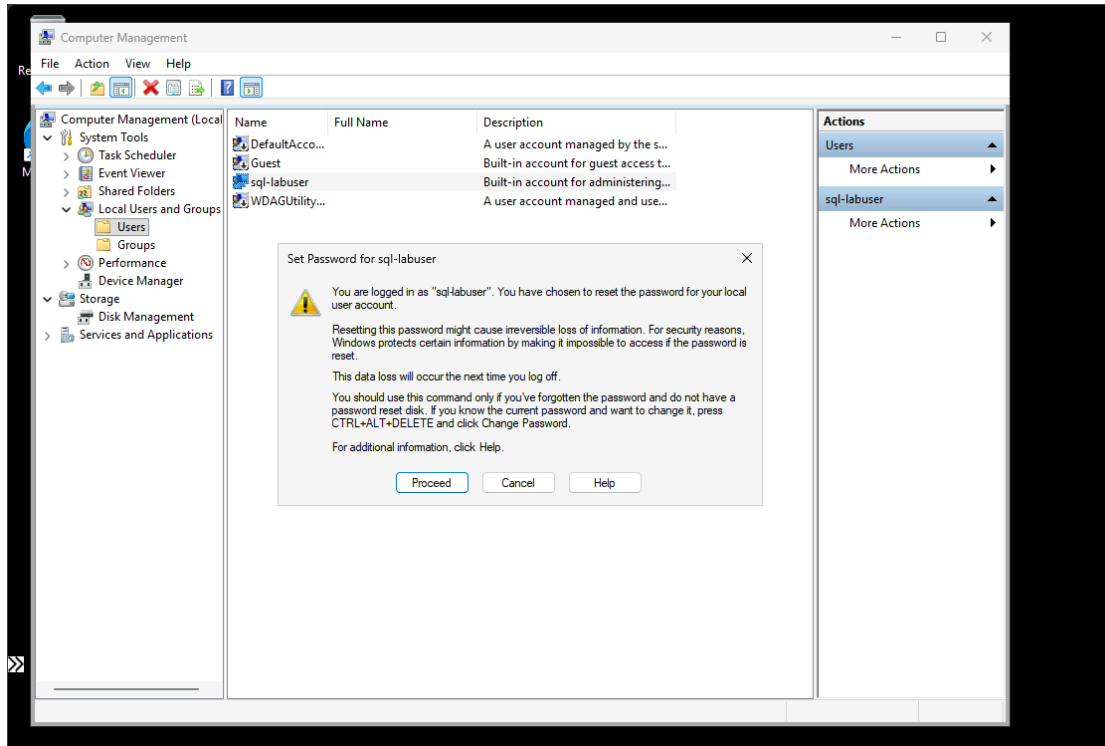
☐ Have no “administrator” account (Delete it)



- ☐ Leave the guest account Disabled (Disable it)



- ☐ Have a strong username/password for your local account



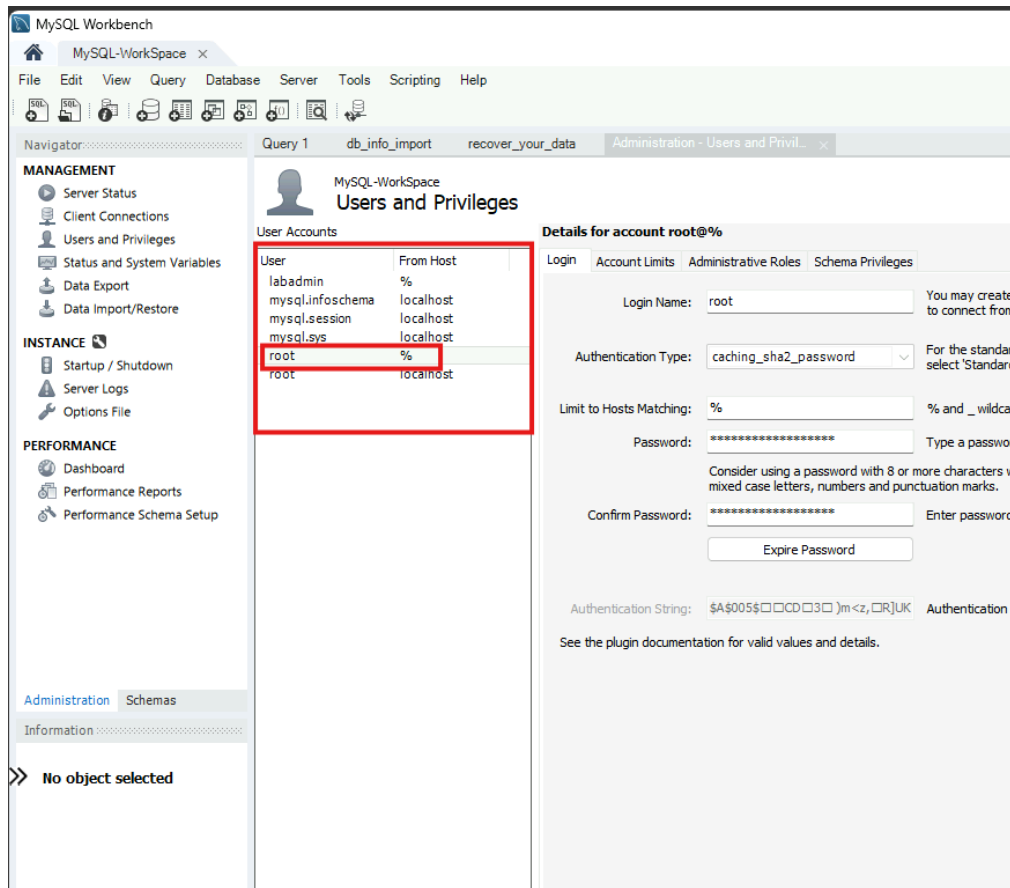
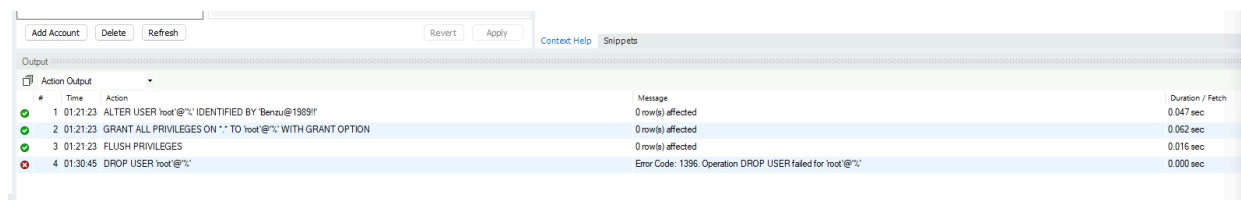
- ☐ Do not allow logon to the MySQL Server from the public internet

```
# SERVER SECTION
# -----
#
# The following options will be read by the MySQL Server. Make sure that
# you have installed the server correctly (see above) so it reads this
# file.
#
[mysqld]
bind-address = 0.0.0.0
```

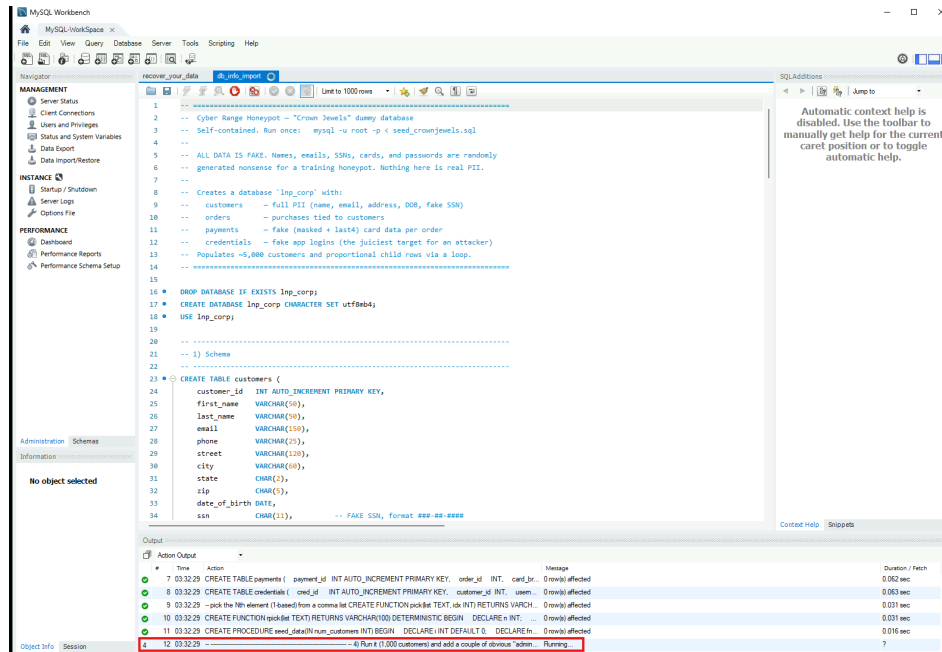
Remove the public IP address from the SQL Server so that it is no longer publicly exposed to the internet.

- ☐ Set a strong root password for the root account that logs in over the network (or delete it)
- ☐ You can run the following SQL command to update the password for the remote **root** account:
 - ☐ Alternatively, you can manually delete the remote **root** account by going back to **Server** → **Users and Privileges** and removing the account configured for network access.

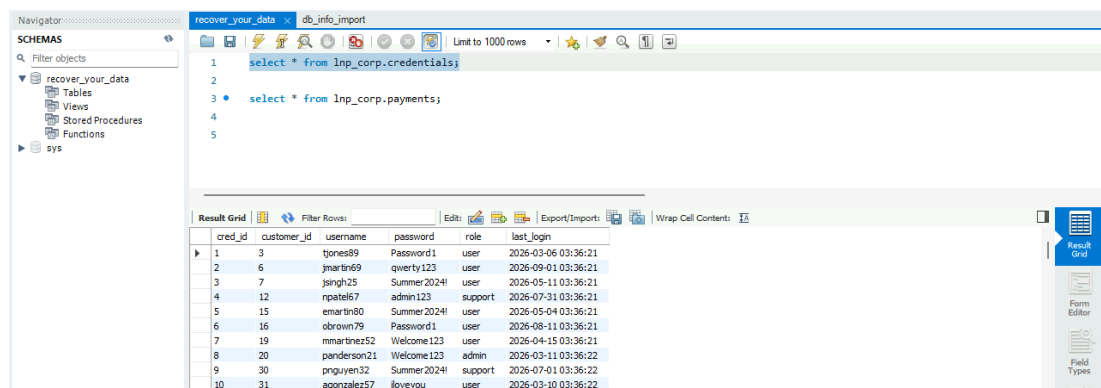
```
ALTER USER 'root'@'%' IDENTIFIED BY 'Benzu@1989!!';
GRANT ALL PRIVILEGES ON *.* TO 'root'@'%' WITH GRANT OPTION;
```

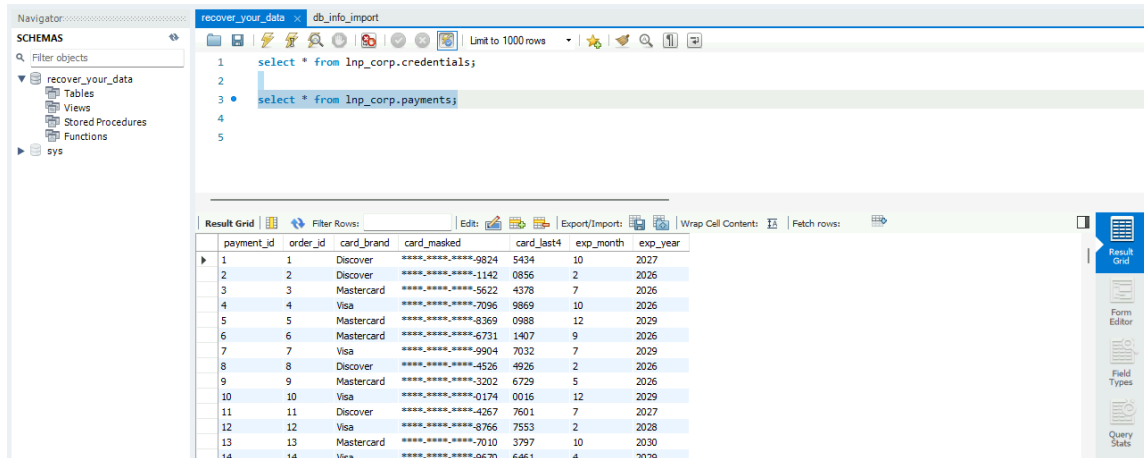

FLUSH PRIVILEGES;**DROP USER 'root'@'%' ;c**

- ☐ “Restore” the data from backup (see [Phase 2 — Install & populate MySQL](#))



Data recover is back on for the sql server after checking the data





☐ Or if you took an actual backup, [actually restore the data from backup](#)

Phase 10 — Reporting

We build our end-to-end incident report.

Defender-Level Investigation

Export the following logs specifically for the time frame from your resource exposure time until isolation

- DeviceLogonEvents
- DeviceProcessEvents
- DeviceRegistryEvents
- DeviceNetworkEvents
- DeviceFileEvents
- MySQLAudit_CL (MySQLAudit_CL_Auth.csv + MySQLAuth_CL_Query.csv)
- NTANetAnalytics (or any other tables you feel are necessary)

Use your favorite LLM (I prefer Claude), upload the files, and use [this prompt](#) to formulate the incident report

Machine-Level Forensic Investigation

Use [this prompt](#) along with your captured investigation packages to see what has changed on the VM, before and after the breach.